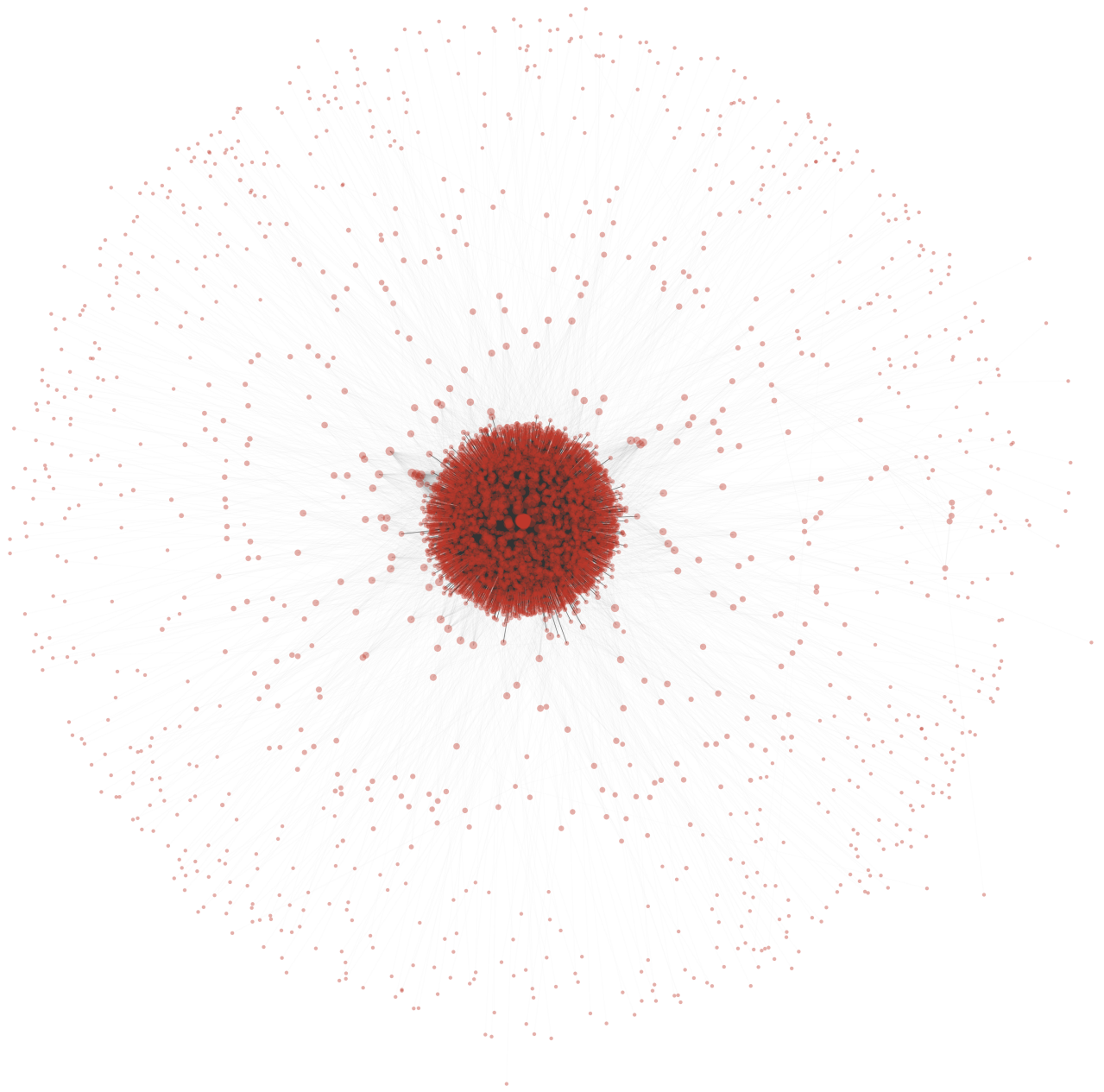


Computer Science

**In Pursuit of an Ethical Framework
for Practical Dark Web Research**

Department of Computer Science
Loughborough University

April 2026



"I'm sorry, Dave. I'm afraid I can't do that."

The crawled graph of a dark web site.

Abstract

Empirical dark web research routinely encounters sensitive and potentially illegal material, and the ethical frameworks currently available to researchers are not well suited to managing the risks this creates. Existing work tends to treat institutional approval as sufficient, leaving vulnerable populations underprotected and researcher safety largely unaddressed. This paper proposes that ethics be treated as a structural property of the research system rather than a procedural requirement, and develops a formal framework on that basis. The framework is verified against two crawler implementations and evaluated empirically, with results suggesting that rigorous ethical constraints need not come at the cost of practical utility.

Contents

1	Introduction	1
2	Literature Review	2
2.1	Scope of Reviewed Literature	3
2.1.1	Methods of Literature Review	3
2.1.2	Gap Analysis	4
3	Ethical Reasoning	9
3.1	Zero-Trust Research	9
3.2	Stakeholder Definitions	10
3.3	Data States and Risks	11
3.4	Safe and Secure Research Environments	12
3.5	Auditability and Documentation	13
3.6	Responsibilities Across the Research Lifecycle	14
4	Verifiable Ethics	17
4.1	Shared Definitions	18
4.2	State Transition Harm Model	19
4.2.1	State Space	19
4.2.2	Transition Structure	20
4.2.3	Harm and Cost	21
4.2.4	Required Properties for the State Transition Harm Model	22
4.2.5	Resultant Arguments from the State Transition Harm Model	23
4.3	Ethical Dependency Graph	24
4.3.1	Decision Structure	24
4.3.2	Graph Structure	25
4.3.3	Exposure and Mitigation	26
4.3.4	Required Properties for the Ethical Dependency Graph	27
4.3.5	Resultant Arguments from the Ethical Dependency Graph	27
4.4	Joint Claims	28
4.5	Weaknesses	29
4.6	Closings on Verifiability	30
5	Ethical Verification in Practice	31
5.1	System Descriptions	31
5.2	Shared Formal Vocabulary	32

5.3	State Transition Harm Model	35
5.4	Ethical Dependency Graph	39
5.5	Joint Claims	41
5.6	Summary	42
6	Implementation	43
6.1	Physical Environment and Access Controls	43
6.2	Network Design and Segmentation	44
6.2.1	VLAN Separation	44
6.2.2	Air-Gapped PXE Boot Server	45
6.3	Version Control and Repository Governance	45
6.4	Operating System Specifications	45
6.4.1	NixOS and Declarative Configuration	45
6.4.2	CIS Benchmark Hardening	46
6.4.3	Encrypted Storage	46
6.4.4	Secure Boot with a Custom Enrolled Key	47
6.5	Software Stack	48
6.5.1	Rust	48
6.5.2	Zeroing Allocator	49
6.5.3	Per-Frame Processing and Immediate Discard	49
6.5.4	TLS and HTTP/2 Fingerprint Spoofing	49
6.5.5	Hyper and HTTP Internals	50
6.5.6	Arti	51
6.5.7	Rate Limiting and Politeness Controls	51
6.5.8	Redirect Following and Redirect Loop Detection	51
6.5.9	SHA-256 URL Hashing	52
6.5.10	Channel-Based Pipeline Architecture	53
6.5.11	OTLP Tracing with Merkle-Signed Spans	53
6.6	Summary of Controls and Framework Mappings	54
6.7	Ethical Costs	54
7	Performance	56
7.1	Experimental Environment	57
7.2	A Note on Crator’s Architecture	57
7.3	Execution Time	58
7.4	Discovered URLs	59
7.5	Pages Per Second	60
7.6	Peak Memory Usage	61

7.7	Comparison with De Pascale et al. (2024)	62
7.8	Interpretations	64
7.8.1	Sources of the Throughput Difference	64
7.8.2	Ethical Overhead	65
7.9	Summary	65
8	Conclusion	66
8.1	Future Work	66
8.2	Conclusion	67
A	Supplementary Tables	72
A.1	Decision Set	73
A.2	Dependency Edges	74
A.3	Reversal Cost Analysis	75
B	Supplementary Diagrams	76
B.1	State Transition Harm Model	77
B.2	Ethical Dependency Graph	78
B.3	Crawled Graph	79
C	Raw Benchmark Results	80
C.1	Execution Time	80
C.2	Discovered URLs	80
C.3	Pages Per Second	81
C.4	Memory Usage	81
C.5	Replicated Execution Time	81
C.6	Replicated Throughput	82

Chapter 1

Introduction

The dark web occupies a paradoxical position in contemporary discourse, being widely praised as a mechanism for protecting free speech and privacy, while also being criticized for enabling serious criminal activity and human rights abuses. As a result, empirical dark web research places researchers in close proximity to highly sensitive material. This includes personal disclosures, criminal marketplaces, and media whose very possession may be illegal. Despite this, existing work offers only limited insight into how researchers protect themselves, their institutions, and the people whose data they collect. Ethical discussions are often brief and procedural, framed in terms of institutional approval or legal compliance, with little detail on how specific risks are actually managed. The Tor network is the most widely used anonymity network and de facto standard for dark web access, attracting the majority of empirical research in this space. It consequently forms the focus of the work presented here.

Consequent to the anonymity that Tor provides, it is relied upon by individuals who may face severe consequences if their activities are revealed, such as political dissidents, journalists, whistleblowers, and abuse survivors. These populations are rarely acknowledged in existing research, which tends to characterise dark web users as either offenders or investigatory targets. Such framing constrains the ethical perspective and leaves significant harms insufficiently examined.

Tor hidden services are disproportionately likely to host illegal or acutely harmful content, creating risks that extend to data subjects, researchers, and the institutions that support the work. The following chapter examines how existing empirical Tor literature articulates ethical concerns, identifying a consistent gap between the sensitivity of the material encountered and the depth of ethical reflection.

Chapter 2

Literature Review

Empirical studies of Tor and related anonymity systems are often conducted in environments where ethical risks are difficult to avoid. Studies of the network’s hidden service landscape have found that illegal or acutely harmful content is disproportionately represented relative to the surface web (Dalins et al., 2018; Guitton, 2013; Moore & Rid, 2016). Researchers may therefore encounter sensitive or otherwise harmful material as part of routine data collection, even when it is not the primary aim of the study. This can expose researchers and other stakeholders to unexpected harm. Tor research therefore makes a useful context for examining how ethical considerations are handled in practice.

Across this literature, ethical considerations are often acknowledged but rarely explored in depth. Many studies include brief statements noting institutional review board (IRB) approval or that the research poses "minimal risk" to participants (De Pascale et al., 2024; Kang & Kim, 2026). These statements indicate procedural compliance, but they do not explain how ethical risks are identified, managed or implemented in the research context.

Instead, ethical approval tends to appear as a procedural checkbox or legitimisation device (Brewer et al., 2021; Rao et al., 2024; Turk et al., 2020), rather than a prospective constraint on research design. As a result the connection between ethical reasoning and methodological choice is often unclear.

This limitation is particularly important in the context of Tor. The network is designed to protect the anonymity of individuals who may face serious consequence if identified. Despite this, existing empirical work rarely treats ethics as something that must be built into the research system itself.

This review therefore focuses on how ethics is invoked and applied in empirical Tor research. The aim is to identify common patterns in current practice and to highlight where existing approaches fall short.

2.1 Scope of Reviewed Literature

This review includes academic works that meet the following criteria:

- They report empirical or practical research involving Tor or closely related anonymity systems.
- May include mention of ethical approval or ethical risk assessment, such as IRB or ethics committee approval, discussion on risk, or statements on legality and acceptability of the research.
- May use ethically sensitive data collection methods, such as the inspection of Tor traffic, crawling hidden services, or deploying infrastructure that would interact with real users and Tor relays.

The review excludes:

- Theoretical, or protocol-design papers that do not involve interaction with live Tor traffic, users or infrastructure.
- General literature on internet or data ethics that do not address Tor or comparable platforms in a substantive manner.
- Papers that mention Tor tangentially and do not use it as a primary data source.

2.1.1 Methods of Literature Review

Various literature databases¹ were searched using a combination of search terms including but not limited to `Tor`, `hidden services`, `dark web`, `crawler`, `scraping`, `ethics`, and `onion`. The search was limited to peer-reviewed work in English, and further relevant literature was found from citations.

After screening titles and abstracts for relevance, each paper was given full text review applying the inclusion and exclusion criteria described above. On this basis, the papers were categorised, and recurring themes were identified.

¹Google Scholar, IEEE Xplore, Web of Science, Science Direct

2.1.2 Gap Analysis

From the analysis of the discovered literature, it is evident that there is a gap in Tor-based research that needs to be crossed to meet robust ethical standards. Existing Tor research commonly includes brief references to ethics, typically in the form of IRB approval (Brewer et al., 2021; Rao et al., 2024; Turk et al., 2020) or assertions that data collection poses minimal risk (De Pascale et al., 2024; Kang & Kim, 2026). Other publications fail to disclose any ethical considerations at all (Bergman & Popov, 2023; Celestini & Guarino, 2017; David et al., 2021; Guitton, 2013; Kang & Kim, 2026; Monterrubio et al., 2021; Park et al., 2018; Pastor-Galindo et al., 2024; Sabbah et al., 2016; Sambath et al., 2024; Xu et al., 2021).

Papers that acknowledge ethical considerations tend to focus on compliance with established guidelines rather than integrating ethics into their methodological decisions. In many cases, ethical considerations appear as a formal requirement rather than as a factor that shapes research design. Turk et al. (2020)’s “A tight scrape: Methodological approaches to cybercrime research data collection in adversarial environments”, for example, references the British Society of Criminology’s ethics statement (British Society of Criminology, 2015) and the Menlo Report (Bailey et al., 2012), highlighting the importance of ethical awareness in cybercrime research. Similarly, Brewer et al. (2021) argue that automated data collection can be conducted ethically, “provided special care is taken by the researcher to acknowledge and explicitly address the complexities surrounding consent, privacy, and a myriad of other potential harms (to subjects, websites, and researchers).” Although some reflection is present in these examples, ethics is positioned as something to be satisfied rather than something that shapes how methods are designed and carried out.

The literature also appears to lack consideration for vulnerable populations and stakeholders, beyond law-enforcement and cybersecurity perspectives. Much of it implicitly frames dark web users as either offenders or investigatory targets (De Pascale et al., 2024; Guitton, 2013; Monterrubio et al., 2021; Xu et al., 2021). Although research indicates that a notable share of the Tor network is associated with illegal content, attention must still be given to the needs of users with legitimate reasons for maintaining anonymity. Guitton (2013)’s “A review of the available content on Tor hidden services: The case against further development” is an extreme violation of this premise. The paper conducts a systematic analysis of Tor hidden services and concludes that “unethical content is quantitatively and qualitatively more preponderant than ethical content,” arguing that support for further development of hidden services should cease. Yet their analysis reproduces forum names, hyperlinks, usernames, and sensitive personal disclosures directly, including content related to sexual trauma. These details are sufficiently specific to be traceable back to individuals or communities. This practice directly conflicts with the basic expectations of privacy and

confidentiality, whilst disregarding the potential harms to individuals whose disclosures were originally made in the expectation of anonymity. In doing so, the research fails to ethically consider the vulnerable populations it identifies. This risks harm to the very individuals its findings are built upon and highlights a critical hypocrisy at the heart of the research.

Likewise, there is a near-complete neglect of risks to researchers themselves. Brewer et al. (2021) is one of the few sources recognising that automated collection of web data in criminology carries risks not only for research subjects and websites, but also for researchers who might encounter illegal or highly distressing content. They note that collecting and analysing media containing child sexual abuse material (CSAM) especially can “cause psychological harm and open researchers up to criminal charges for accessing and possession,” and that even textual depictions of serious victimisation can be distressing. Researchers may also face reporting obligations or reprisals if their presence becomes known. Such concerns are, however, rarely discussed elsewhere in the literature.

These risks are evident in the work of Dalins et al. (2018), who obtained governmental approval to conduct an unrestricted crawl of Tor hidden services and explicitly stored illegal material, including CSAM. While they describe strong technical measures to protect the collected data, they provide little information on researcher access or protocols for managing exposure to distressing content. This omission may reflect the common practice of delegating responsibility for researcher safety to the host institution rather than addressing it within the study itself.

This pattern appears across the broader corpus. Many empirical works make little or no mention of researcher risk (Bergman & Popov, 2023; Celestini & Guarino, 2017; David et al., 2021; De Pascale et al., 2024; Guitton, 2013; Kang & Kim, 2026; Monterrubio et al., 2021; Park et al., 2018; Pastor-Galindo et al., 2024; Sabbah et al., 2016; Sambath et al., 2024; Xu et al., 2021). Where ethical consideration exists, it typically focuses on subject-level risk, with little regard for researcher wellbeing.

A further recurring issue is the conflation of legal and ethical acceptability. Several papers justify their methods primarily by pointing to legal permissibility, the public or semi-public nature of data, or the fact that they have obtained institutional clearance. Dalins et al. (2018) highlight governmental authorisation to collect illegal material; Pastor-Galindo et al. (2023) describe following the Tor Project’s own ethical research guidelines on the gathering of onion addresses; and other authors emphasise that only “publicly accessible” or “non-personal” information is examined (Kang & Kim, 2026; Kuhn et al., 2024). Rao et al. (2024) explicitly state that research on dark web marketplaces raises “unique ethical and legal challenges” and stress the importance of complying with applicable laws and institutional guidelines. Nevertheless they still implement a crawler that stores raw HTML from marketplaces in basic databases without detailed discussion of data minimisation, encryption,

access control or retention. As “The Menlo Report” (Bailey et al., 2012), and Brewer et al. (2021) both stress, legality and public accessibility are not sufficient to establish ethical permissibility. Some legally permissible practices — such as publishing identifiable quotes from trauma disclosures (Guitton, 2013) or publicly releasing crawling code (Bergman & Popov, 2023; De Pascale et al., 2024; Kuhn et al., 2024; Pastor-Galindo et al., 2023) — may still post serious ethical problems. Conversely, some ethically defensible research designs may challenge existing legal boundaries. Substantive ethics requires a clear distinction between what is legal and what is morally justified in context.

These conceptual gaps are reinforced by the way many papers treat the operational environment of their systems. In general, there is little information about how sensitive data are stored, and how access is managed. Only a single paper in the identified corpus, Dalins et al. (2018), offers detailed information about data storage, describing a datastore “secured with SSL/TLS using random passwords, upon a RAID-10 array encrypted using LUKS,” with remote access limited via firewall rules and long random passwords. This indicates some attention to confidentiality and integrity. By contrast, many other studies either give no information about storage at all (Celestini & Guarino, 2017; Monterrubio et al., 2021; Xu et al., 2021) or briefly mention that data are stored in a local (Monterrubio et al., 2021; Sambath et al., 2024; Xu et al., 2021) or cloud-based database (Park et al., 2018; Pastor-Galindo et al., 2024). Monterrubio et al. (2021) warrants specific attention here, as the research appears to have been run on the researcher’s personal laptop, reflecting a notable absence of consideration for the risks that handling illegal content on a personal device would present.

Furthermore, several frameworks and crawlers are implemented on distributed cloud platforms to maximise scalability and performance (Park et al., 2018; Pastor-Galindo et al., 2024; Ródenas et al., 2024), but the ethical and legal implications of handling sensitive or illicit material through third-party cloud providers are not explored. When researchers delegate data storage and processing to commercial providers, questions arise around jurisdictional authority over seized content and contractual obligations. Of equal concern is the risk that providers may disclose data in ways that fall outside what the original researchers expected.

In some cases, the code for Tor crawlers or monitoring systems is released openly (Bergman & Popov, 2023; De Pascale et al., 2024; Kuhn et al., 2024; Pastor-Galindo et al., 2023), which may enable re-use for purposes not anticipated by the original authors. Ródenas et al. (2024) note that the explicit publication of any such framework “might inadvertently aid malicious actors,” and therefore propose a “controlled release strategy” limited to researchers and law enforcement.

Within the broader landscape, the DICE-E framework (Darknet Identification, Collec-

tion, Evaluation, with Ethics) represents one of the few attempts to offer a structured, domain-specific guide for Darknet research (Benjamin et al., 2019). DICE-E is motivated by the observation that “society’s growing dependence on computers and information technologies has been matched by an escalation of the frequency and sophistication of cyber attacks committed by criminals operating from the Darknet,” and that many scholars “lack the capability or expertise to operationalize Darknet research and are thus unable to contribute to this increasingly impactful body of literature.” The framework sets out four stages for Darknet forum research: “(1) identification of Darknet data sources, (2) data collection strategies, (3) evaluation of Darknet data, and (4) ethical concerns related to Darknet research”, and is explicitly intended as an accessible reference point for scholars entering the field. It offers guidance on handling cybercriminal assets unique to the Darknet, but explicitly notes that it “is not intended to provide an exhaustive listing of all possible directions that can be taken, nor is it meant to be an in-depth discussion of the methodologies themselves,” instead serving “as an introductory point for researchers to begin thinking of research questions and analyses that possibly could be explored with Darknet data.” As such, DICE-E usefully foregrounds ethical concerns as an integral part of the research workflow, but it does not fully develop a substantive analysis of ethical considerations. This is evident in Kuhn et al. (2024)’s paper, which cites DICE-E and usefully identifies relevant stakeholders, but provides little meaningful detail on how this affected their data collection and retention strategies.

Despite these shortcomings, a small number of works offer partial exemplars of more substantive engagement with ethics. As mentioned, Dalins et al. (2018) obtained explicit permission to collect and store illegal content and set out well structured data protection policies. This is a strong response to the legal and safety risks that come with handling illicit material, and it shows that governance and security were well-reasoned about. Their work is clearly oriented towards assisting law enforcement in prioritising cases, which may be a large motivating factor for these measures. Kuhn et al. (2024) also draw clear ethical boundaries. They avoid interacting with dark web communities and operators, which reduces the risk of researchers influencing behaviour or causing harm through engagement. They restrict data collection to information that is publicly available without authentication, which avoids bypassing access controls and reduces the likelihood of collecting material that users shared in more limited settings. They also refrain from publicly releasing datasets which reflects their attention to the potential misconstruances of research data. Kang and Kim (2026) maintain similar commitments to Kuhn et al. (2024) whilst additionally stating that "collected data were stored in encrypted repositories with access restricted to authorized researchers and were used exclusively for academic research purposes." Conversely Brewer et al. (2021) provide a criminological framework for web crawling and scraping ethics, in

which they address consent, privacy, stakeholder harm, and reporting requirements. These contributions illustrate what more field-specific ethical engagement might look like, even if they do not extend to all relevant scenarios.

Overall, existing Tor research overwhelmingly adopts a procedural conception of ethics, often equating ethical adequacy with institutional approval and legality. Although we might hope these studies adhere to ethically reasonable standards, the morally responsible default is to assume that they do not. Without in-depth thought about the underlying ethical decisions, researchers could unintentionally cause harm to unsuspecting stakeholders. In neglecting this, researchers risk undermining user privacy and exposing them to new harms.

This paper advocates for the security and ethical precautions used by researchers to be systematically outlined in a manner that promotes reproducibility and accountability across the research community. Such transparency enables studies to be conducted in a manner that protects researchers, institutions, and vulnerable populations who may require anonymised communication. Building on the patterns identified in this literature review, we propose a framework that promotes the safety and dignity of all involved parties that can be adopted by future Tor research.

Chapter 3

Ethical Reasoning

As the literature review established, the ethical shortcomings of existing dark web research are not primarily failures of intention but of architecture. Researchers operate near acutely sensitive material, yet the frameworks available to them locate ethical responsibility in approval processes and individual conduct rather than in the systems through which data flows. This chapter builds directly on the gaps identified in Chapter 2, moving from procedural ethics towards a framework that can shape research design.

3.1 Zero-Trust Research

The standard procedural approach implicitly assumes the researcher is a trusted actor who simply needs guidance on compliance. In the context of the dark web, this is not an adequate foundation. In this environment, researchers may be placed in proximity to highly sensitive material which may be under the inherent expectation of anonymity. Exposure to this material carries risks not only for the researcher, but for those in their immediate environment. An unattended screen or unsecured device can inadvertently expose bystanders to deeply harmful content without the researcher’s awareness. Under these conditions, “trusting the researcher” becomes a fragile safety assumption, with even small lapses in judgement causing harms that are irreversible and difficult to detect.

We therefore adopt a zero-trust framing for this paper. Zero trust is traditionally used in information security as a design approach where no user, device, or network segment is assumed trustworthy by default, even when it is inside an organisational boundary. Access is granted only as needed and is continuously constrained and verified, rather than being implied by role, location, or good faith. In this context, zero trust does not claim that researchers or institutions are unethical. It recognises that ethical outcomes cannot depend

on individual discretion or continuous oversight. Researchers can make mistakes, oversight can be partial, and external pressures can arise during a project. A zero trust approach treats these as normal conditions of real research and responds by requiring technical and procedural constraints that limit what any stakeholder can access, store, or disclose. This makes the framework enforceable in practice rather than reliant on a researchers intentions.

This zero trust posture is also necessary because the risks surrounding Tor research do not end with the immediate research team. Tor research systems and artefacts can become attractive targets and tools once they exist. Researchers themselves may seek to access collected material beyond the approved scope, while third parties may attempt to compromise researchers or their infrastructure to obtain data or identify the focus of their work. The publication or sharing of tooling can also enable misuse by external actors. This includes governments, foreign agents, and other parties who may repurpose research software for surveillance or other operational aims that do not align with the original intent of the work. Even where such use is lawful within a given jurisdiction, it may still violate the ethical commitments under which the data was collected, or expose Tor users and service operators to harms that the research sought to avoid. These possibilities reinforce the need for strict constraints on access, retention, and disclosure, and for careful decisions about what is released and what must remain controlled.

3.2 Stakeholder Definitions

Ethical Tor research must rigorously define every stakeholder that may be affected, going beyond generic references to "participants" or "users." Stakeholders must be tied to specific data flows and technical decisions, because the same research action can carry very different consequences depending on who is affected and how.

The most directly exposed group are Tor participants themselves. This includes users and operators of hidden services, encompassing both offenders and vulnerable populations such as political dissidents, journalists, and abuse survivors. These actors generate the content and interactions that researchers collect. They bear the consequences most immediately if deanonymisation occurs, or if research infrastructure inadvertently increases surveillance on the services they depend on.

Not all affected individuals are Tor users. Others are captured incidentally, having no knowledge of or connection to the network whatsoever. Victims of doxxing or named third parties in leaked documents may find their privacy and safety compromised by decisions made at any stage of the research process. These individuals have no awareness of the research and no meaningful way to consent to it, which makes their position particularly difficult to account for within standard ethical frameworks.

Researchers and research teams are stakeholders in their own right. They design and operate the systems through which data flows, and in doing so they may expose themselves to material that is illegal, deeply disturbing, or both. Legal liability for possession of such content is a concern that must be considered. So too is the risk of reprisal if their work becomes visible to hostile groups.

Institutions and oversight bodies carry a different but equally serious set of interests. Universities, ethics committees, and data protection units provide the infrastructure through which research is conducted and are ultimately accountable for its compliance with legal and professional standards. This accountability is not merely formal. If research practices are later challenged, these bodies may face reputational damage and direct legal liability.

Finally, external actors are affected by the publication of Tor research in ways that are harder to anticipate. Law enforcement and regulatory bodies may seek access to tools or datasets that a study has produced, and published findings can support better targeted investigations. However, releasing detailed infrastructure descriptions or address lists can expand surveillance capabilities well beyond what the original study intended. Institutions may also face pressure to disclose data in ways that exceed prior ethical commitments. External actors must therefore be recognised as stakeholders whose interests are considered but not prioritised, and whose potential misuse of research outputs must inform decisions about what is retained and published.

3.3 Data States and Risks

Tor-derived data moves continuously through a research system, and ethical risks are present at every stage. The Tor network provides strong protections while data is in transit. Layered encryption and relay-based routing prevent intermediaries from observing both origin and destination. These protections, however, end once data leaves the network. Transfers between system components and storage in databases or caches can all create unintended persistence or exposure. Data in motion often becomes data at rest, and every transition introduces potential harm to stakeholders.

When material is stored, risks increase. Persistent storage allows searching, copying, and long-term retention, which can amplify harm if the content is sensitive or illegal. Identifiers, unique text fragments, or forum metadata can be combined to deanonymize individuals, and retained data remains vulnerable to future deanonymization as new information or techniques emerge. In small communities this linkability is particularly acute, and the literature shows how reproducing specific names or quotes in publication compounds the problem further. Retained datasets also invite scope drift. A dataset that exists will attract new questions and new uses, and there is no reliable way to constrain this through intention

alone. Legal and institutional exposure compounds the risk further still. Retaining illegal material carries personal and organisational liability regardless of research intent (Dalins et al., 2018).

Movement itself also carries risks that are easy to overlook. Transferring sensitive material across networks or providers can bring it under different legal jurisdictions and institutional policies. Timing information, transfer volumes, and target addresses can cause harm when combined with other available data, even when the payload itself is protected. The ethical default must therefore be to treat data in transit as though it can become data at rest at any moment, and to constrain it accordingly.

Human interaction with data introduces further risk. Researchers may encounter potentially illegal or deeply distressing content during what appears to be routine work. A debugging step or sanity check may not feel significant, but they can produce durable artefacts that bring raw content into the human-accessible environment (Posey et al., 2011). Exposure to disturbing material causes psychological harm, and exposure to illegal content creates personal liability (Dalins et al., 2018). Even a well-governed project can be compromised by an individual with unprotected access, and it is not enough to assume that a team will remain disciplined. The system itself must make discipline enforceable. This is why any credible framework must ensure that humans only ever encounter constrained, derived outputs, and that all exceptions remain auditable.

The purpose of naming these states is to make transitions between them visible and governable. Systems drift from transit to rest through caching and logging, and publication can function as an irreversible transition that releases sensitive material into the open. Ethical claims must therefore be expressed in terms of what data is permitted to exist in each state, and what technical controls prevent prohibited transitions. A framework that cannot demonstrate that raw content was never retained or viewed collapses back into trust, which this paper rejects as an adequate ethical control.

3.4 Safe and Secure Research Environments

Creating a safe and secure research environment should not be a peripheral implementation detail in Tor research. It should be a primary ethical control. This follows directly from the zero-trust posture described above. If ethical outcomes cannot rest on researcher discretion alone, then the environment should make it difficult to access sensitive material, difficult to retain it unintentionally, and straightforward to demonstrate that approved constraints were followed.

Operational security (OpSec) is therefore of the utmost priority because it is how a zero-trust framework becomes enforceable. As soon as operator access becomes normalised,

the framework dissolves into reliance on individual discretion. By contrast, a hardened and monitored environment with strict boundaries and minimal privileges makes Tor derived data inherently more secure by default.

This is also why physical security must be treated as coequal with digital security. The threat model is not limited to remote compromise. It includes unauthorised local access and loss or theft of equipment. If an adversary can physically access a system that handles Tor derived data, many digital controls become much easier to bypass.

Institutional ownership of the environment also matters because it makes compliance demonstrable. If sensitive Tor derived material is stored on a researcher’s personal device (Monterrubio et al., 2021), institutional audit trails are effectively impossible. In these cases, liability becomes ambiguous and destruction of sensitive materials cannot be verified. If that material is instead handled on institutionally controlled systems (within a physically secured context) access can be constrained and recorded, and researchers can provide auditable evidence that the system operated within approved ethical boundaries.

Researchers should always acknowledge the risk that they may be put under legal scrutiny simply by virtue of their work, even when operating under institutional approvals. Approval reduces risk, but it does not eliminate the possibility of investigation, since the material encountered may include content that is illegal, disturbing, or linked to active criminal activity. It also does not prevent third parties from seeking access to research artefacts. This is precisely why physical security, strict access control, and clear institutional ownership of the research environment must be treated as part of the ethical framework. They protect research subjects by limiting exposure and retention, and they protect researchers and institutions by ensuring that if scrutiny occurs there is a defensible record showing that the system was operated within approved constraints.

3.5 Auditability and Documentation

A framework is only as strong as its ability to demonstrate compliance. Given the sensitivity of this work, it is not sufficient to claim that protections were in place, there must be a traceable record of what was actually done.

Auditability in this setting must be understood as more than “logs exist.” A log can show that a process ran, that a request was made, or that a user authenticated. It does not, on its own, establish that the system respected the ethical perimeter that justified the research in the first place. Under a zero-trust framing, the research environment must therefore be constructed such that both the movement of Tor-derived material and the decisions that govern that movement remain reconstructable after the fact.

Crucially, this requirement extends beyond software. Auditability must cover the re-

search pipeline as a whole, across the full lifetime of the project. Research is not a single act of collection and analysis. It is a chain of decisions that begins from the conception of an idea and until completion. If these decisions cannot be reconstructed, then ethical compliance is once again dependent on intent.

A useful way to frame this is that auditability is evidence of governance. It should be possible to show, at any later point, all approval and each step of ethical and practical decision making. This includes the reasoning that justified these steps. Ethical and technical decisions should be treated as ethical artefacts because they determine what harms are possible in practice.

The audit record must also be designed with restraint in mind. The system must generate enough traceability to show compliance, but not so much that the audit trail itself becomes an attractive or dangerous dataset. Auditing sources can quietly create new stores of sensitive material with weaker controls than the primary system. Researchers, therefore, must exhibit discipline in what is recorded and how it is stored. The objective being to preserve accountability for actions and decisions, not to preserve the underlying content.

Auditability must also include change control and operational provenance. Code and configuration deployments must be traceable, and it must be possible to reconstruct the non-technical decisions that shape the project. Technical sophistication alone is no guarantee of ethical transparency. Consequently, it is the researcher’s responsibility to provide documentation that backs all of these claims.

For these reasons, transparency and auditability should be treated as mandatory. They protect affected parties by making claims about minimisation and non-exposure testable, and protect researchers and institutions by providing a contemporaneous record that can withstand scrutiny.

3.6 Responsibilities Across the Research Lifecycle

Ethical responsibility in Tor research is frequently treated as a discrete concern, something addressed at the point of data collection or resolved through institutional approval before work begins. This paper argues that ethical consideration is warranted at every stage, since harms can emerge from otherwise routine activity at any point.

Ethical reasoning must begin at the point where a research question is first scoped. The choices made here, about objectives, methods, and feasibility, carry consequences for every stakeholder group identified in the preceding section. A useful discipline at this stage is to make the required data explicit and to ask what can be avoided. Where an objective is achievable through aggregation or short-lived processing, collecting raw content requires additional justification, since it increases retention risk, linkability, and the potential for

unintended exposure.

As scoping gives way to planning, the focus shifts toward governance and operational trust boundaries. Institutional review and legal consultation may be necessary, but they do not communicate how a project intends to manage the practical risks that arise during live work. Broad commitments, such as those related to minimisation and non-interaction, should be made explicit with clear specification. Foreseeable disruptions should also be anticipated, including encounters with illegal material or external requests for disclosure. Even where such plans cannot cover every contingency, they reduce reliance on ad hoc judgement and provide evidence that risk management was considered before work began.

Implementation is the stage at which ethical aims are most readily undermined. Unplanned persistence can place Tor-derived data into incidental human view during development and testing. System architecture should therefore be treated as a primary ethical control, not a neutral implementation detail. Designs that transform raw content into constrained derived outputs early in the pipeline, and that limit what is written to persistent storage, reduce both stakeholder exposure and researcher risk. The fewer unnecessary transitions into data-at-rest and data-in-use, the fewer points at which harm can occur.

Once operation begins, ethical responsibility becomes a matter of maintaining the boundaries set during planning. Idealised commitments can drift from their planned boundaries, especially in adversarial environments. Retention limits are most credible when they are technically enforced rather than manually applied, since manual processes are likely to be applied inconsistently across long executions and fluctuating institutional control.

Data analysis introduces its own concerns. Outputs that appear non-identifying in isolation can become linkable when combined, and illustrative examples can remain traceable even when explicit names are omitted. A cautious approach prioritises aggregation and derived representations, treating any identifying or traceable content as a potential risk. Where manual inspection of collected material is necessary for quality assurance, it should be conducted against locally sandboxed or synthetic datasets rather than raw collected material.

Publication must be understood as a form of disclosure that is difficult to reverse. Releasing addresses, identifiers, verbatim excerpts, or detailed architectural descriptions can increase the risk of tracing or misuse by third parties. A project can be transparent about its constraints and governance decisions without releasing artefacts that enable downstream harm. Where sharing code or data is considered important for scientific scrutiny, a controlled release strategy, as proposed by Ródenas et al. (2024), offers a reasonable compromise between accountability and harm reduction.

Ethical responsibility does not end when a project concludes. Retained artefacts continue to create exposure, particularly when staff move on and the original context is lost. A

planned retention and destruction schedule, specifying what must be kept for accountability and what must be destroyed, is therefore part of the ethical commitment.

Across all of these stages, this paper proposes a consistent orientation rather than a fixed rule set. Ethical commitments should function as practical constraints on what a system is designed to permit. This framing does not remove the need for judgement, but it makes ethical reasoning visible across the full lifecycle of a project and, importantly, makes it something that can be demonstrated rather than asserted.

Chapter 4

Verifiable Ethics

Defining a rigorous ethical framework for Tor research requires more than a set of self-imposed principles. It requires formal tools that make ethical claims testable through construction rather than intent. To this end, this chapter introduces two complementary formal methodologies: The State Transition Harm Model and the Ethical Dependency Graph. Both models take inspiration from other ethical modelling systems such as the Bell-LaPadula model (LaPadula & Bell, 1996), Denning Lattices (Denning, 1976), and typical data lifecycles.

The State Transition Harm Model (STHM) treats the research system itself as a unit of ethical analysis. It defines the states a data artefact can occupy and how data transitions between these states. Controls are then used to block or permit specific transitions. By doing so, we can produce structural guarantees about what a system can and cannot do with Tor-derived data. These guarantees should hold independent of researcher behaviour, because they are properties of the system’s architecture rather than the people operating it.

The Ethical Dependency Graph (EDG) considers the line of reasoning behind a system as the focus of analysis. It maps the dependencies between design decisions and traces how ethical exposure propagates through the research lifecycle. This structure produces a justifactory record that makes reasoning behind architectural choices explicit. It also guards against the kind of circular, post-hoc rationalisation that the literature review identifies in existing Tor research.

These two methodologies are not mutually exclusive. They are designed to work together to balance each other’s weaknesses. The STHM aims to prove that certain harms are structurally impossible, whilst the EDG proves that this impossibility was a deliberate and sensible choice. Without both, a system could be structurally safe by coincidence or well-reasoned but ultimately unsound. Together, they provide both the justification and the proof of a system’s ethical validity.

4.1 Shared Definitions

Definition 4.1 (Stakeholder Set). A finite set $\mathcal{S}$ of stakeholder classes whose interests may be affected by the research system.

$$\mathcal{S} = \{s_P, s_I, s_R, s_N, s_X\}$$

denoting direct tor participants, indirectly represented individuals, researchers, institutions, and external actors respectively.

The stakeholder set identifies whose interests the framework must account for. Every formal claim made about harm or safety is ultimately a claim about one or more of these groups.

Definition 4.2 (Harm Vector). A function $\kappa : \mathcal{S} \rightarrow \{0, 1, 2, 3\}$ which assigns a harm severity to each stakeholder, where 0 denotes no harm, 1 low, 2 moderate and 3 high. The set of all harm vectors is $\mathcal{K} = \{0, 1, 2, 3\}^{|\mathcal{S}|}$. The zero vector $\mathbf{0} \in \mathcal{K}$ assigns no harm to any stakeholder.

The harm vector is how severity is recorded. Rather than treating harm as a single value, it is tracked per stakeholder, so a transition that endangers Tor participants but poses no risk to the institution is represented distinctly from a risk that affects both.

Definition 4.3 (Control Set). A finite set $\mathcal{C} = \mathcal{C}_T \cup \mathcal{C}_P$ separated into technically enforced controls $\mathcal{C}_T$ and policy controls $\mathcal{C}_P$. Under zero-trust, only $\mathcal{C}_T$ is treated as reliable. A control in $\mathcal{C}_T$ cannot be bypassed without modifying the system itself. A control in $\mathcal{C}_P$ depends on human compliance and is therefore insufficient in the context this paper.

The distinction between technical and policy controls is fundamental to the zero-trust approach put forward by this paper. A policy control, such as a researcher agreeing not to retain data, is only as reliable as the person who agreed. A technical control, such as a system that is architecturally incapable of writing to disk, holds regardless of researcher behaviour.

4.2 State Transition Harm Model

4.2.1 State Space

Definition 4.4 (Property Set). A finite, project specific set $\mathcal{P}$ of binary properties of a data artefact. The choice of $\mathcal{P}$ is specific to the research context and must be declared during the planning phase.

Properties are the vocabulary the model uses to describe a data artefact. Choosing them carefully is a research obligation. A poorly chosen property set produces a model that is formally valid, but blind to real risks.

Definition 4.5 (State). A valuation $q : \mathcal{P} \rightarrow \{0, 1\}$ assigning each property a truth value. The state space is $\mathcal{Q} = \{0, 1\}^{|\mathcal{P}|}$.

Each state captures the full condition of a data artefact at a given moment, encoding which protections are currently active.

Definition 4.6 (Protective Properties). A declared subset $\mathcal{P}^+ \subseteq \mathcal{P}$ of properties whose presence indicates strong protection of data artefacts. Define the protection order $\sqsubseteq$ on $\mathcal{Q}$:

$$q \sqsubseteq q' \iff \forall p \in \mathcal{P}^+ : q(p) = 1 \implies q'(p) = 1$$

This defines a formal ordering over states based on how well-protected an artefact is. A state is considered at least as protective as another if it preserves every property that the other does. Intuitively, this ordering is what allows the model to later require that the system never moves data to a lower state of protection. This is similar to the no-write-down principle in the Bell-LaPadula model (LaPadula & Bell, 1996).

Definition 4.7 (Degenerate States). A state $q \in \mathcal{Q}$ is degenerate if it represents an unsatisfied combination of properties. Degeneracy is declared as a set of axioms over $\mathcal{P}$ of the form $p_i = 1 \implies p_j = 0$, which describes incompatibilities between properties. The admissible state space is $\mathcal{Q}^* = \mathcal{Q} \setminus \Delta$ where Δ is the set of states which violate at least one declared axiom.

Some combinations of properties are contradictory by design. A data artefact cannot simultaneously be encrypted and stored in plaintext. Degenerate states are simply those in which such a contradiction holds.

Definition 4.8 (Forbidden States). A declared subset $F \subset \mathcal{Q}^*$ of admissible states that must never be reached. Each element of F is a set of property assignments $\{p_i = v_i\}$ that characterise an unacceptable condition.

Forbidden states define what the system must never allow. Declaring them explicitly is what allows the model to later prove that certain harms are impossible.

4.2.2 Transition Structure

Definition 4.9 (Trigger Set). A finite set Λ of events that can cause state transitions. Triggers must be declared during planning and must cover both intentional operations and involuntary events such as leaks and breaches.

For the case of this paper, we define the trigger set as follows:

$$\Lambda = \{\lambda_{\text{recv}}, \lambda_{\text{buf}}, \lambda_{\text{write}}, \lambda_{\text{xform}}, \lambda_{\text{view}}, \lambda_{\text{pub}}, \lambda_{\text{del}}, \lambda_{\text{leak}}, \lambda_{\text{breach}}\}$$

Denoting receiving from network, buffer in memory, write to storage, transform to derived form, expose to human view, publish externally, delete, unintentionally leak, and security breaches respectively. The final two triggers are involuntary and cannot be prevented by policy controls alone.

Triggers are the events that can move data from one state to another. Importantly, the trigger set must include events that are not under researcher control. A leak or a breach is not something a researcher chooses, but it is something that a researcher can account for.

Definition 4.10 (Transition System). The transition harm model is a tuple $\mathcal{M}$ of definition

$$\mathcal{M} = (\mathcal{Q}^*, Q_0, \Lambda, T, \kappa, \mathcal{A})$$

Where $Q_0 \subseteq \mathcal{Q}^*$ is the set of initial states in which data enters the research system, $T \subseteq \mathcal{Q}^* \times \Lambda \times \mathcal{Q}^*$ is the transition relation written as $q \xrightarrow{\lambda} q'$, $\kappa : T \rightarrow \mathcal{K}$ assigns harm vectors and $\mathcal{A} : T \rightarrow 2^{\mathcal{C}}$.

The transition system connects everything defined so far. States, initial conditions, events, harms, and controls are brought together into a single structure that can be reasoned about formally.

Definition 4.11 (Reachability). A state $q' \in \mathcal{Q}^*$ is reachable from Q_0 , written $q' \in \text{Reach}(\mathcal{M})$, *iff* there exists a finite sequence:

$$q_0 \xrightarrow{\lambda_1} q_1 \xrightarrow{\lambda_2} \dots \xrightarrow{\lambda_n} q_n = q'$$

Where $q_0 \in Q_0$ and each transition is in T .

Reachability determines which states the system can actually arrive at. If a forbidden state is unreachable, the system cannot enter it by design.

Definition 4.12 (Blocking). A transition e is blocked *iff* $\mathcal{A} \cap \mathcal{C}_T \neq \emptyset$. The zero-trust reachable set $\text{Reach}_T(\mathcal{M})$ is computed over only unblocked transitions ($\mathcal{A} \cap \mathcal{C}_T = \emptyset$). If a value is in $\text{Reach}_T(\mathcal{M})$, then it is implied that the state is reachable in the case that all policy controls fail concurrently.

A blocked transition is one that a technical control prevents from occurring. The zero-trust reachable set describes worst-case exposure if every policy control failed simultaneously. In other words: what state could the system reach if all human controls broke down at once? If no forbidden state appears in this set, then a system is safe independent of researcher behaviour.

Definition 4.13 (State Exclusivity). $\mathcal{M}$ satisfies state exclusivity *iff* every artefact occupies exactly one state at any point in time and all transitions are atomic.

$$\forall a, \forall t, \exists! q \in \mathcal{Q}^* : \text{state}(a, t) = q$$

This ensures that concurrent states cannot exist which could exponentially increase the potential state transitions.

State exclusivity prevents an artefact from existing in two states at once, requiring all transitions to be atomic. Without this, harm calculations become approximate and the worst-case bound loses its meaning.

4.2.3 Harm and Cost

Definition 4.14 (Path Harm). For execution path $\pi = q_0 \xrightarrow{\lambda_1} \dots \xrightarrow{\lambda_n} q_n$ the path harm is:

$$K(\pi) = \bigoplus_{i=1}^n \kappa(q_{i-1} \xrightarrow{\lambda_i} q_i)$$

Where $\oplus$ denotes componentwise maximum. Maximum is used rather than summation because harm to a stakeholder is dominated by the worst exposure along the path rather than compounding.

Definition 4.15 (Reachable Harm). The reachable harm of $\mathcal{M}$ is:

$$\mathcal{H}(\mathcal{M}) = \bigoplus_{\pi \in \text{Paths}(\mathcal{M})} K(\pi)$$

Ethical cost measures the residual harm that the system permits by design. The improvement figure gives a quantitative basis for comparing a constrained system against an unconstrained baseline. This paper requires a positive improvement across every stakeholder dimension before a design can be considered ethically justified. representing the worst-case harm vector across all possible executions. Under state exclusivity (Definition 4.13), this bound is exact rather than approximate (Corollary 4.3).

Definition 4.16 (Ethical Cost). The ethical cost of $\mathcal{M}$ is:

$$\Gamma(\mathcal{M}) = \sum_{e \in E_P} \kappa(e)$$

where E_P is the set of unblockable permitted transitions and summation is componentwise. The ethical improvement of an unconstrained baseline $\mathcal{M}_{\text{naive}}$ is:

$$\Delta\Gamma = \Gamma(\mathcal{M}_{\text{naive}}) - \Gamma(\mathcal{M})$$

Ethical cost is the residual harm the system permits by design. A design is only considered ethically justified if it yields a positive improvement across every stakeholder dimension.

4.2.4 Required Properties for the State Transition Harm Model

To meet the requirements laid out in previous sections, we require a set of properties to be valid in order for our needs to be met. Each of these respective properties implies their following statement.

$$\text{Reach}_T(\mathcal{M}) \cap F = \emptyset \quad (\mathbf{Zero-Trust\ Safety})$$

No forbidden state is reachable under the assumption that all policy controls have failed.

$$\forall (q \xrightarrow{\lambda} q') \in T, q \sqsubseteq q' \quad (\mathbf{Protection\ Monotonicity})$$

No transition can reduce the level of protection applied to a data artefact. Once data enters a well-governed state it cannot move to a less protected one.

$$\forall q \in \text{Reach}(\mathcal{M}), \exists \text{path } q \rightarrow q_\omega \quad (\mathbf{Liveness})$$

Every reachable state has a path leading to verified destruction of the data artefact. This implies that data cannot be retained indefinitely.

$$\Delta\Gamma > 0 \quad (\mathbf{Strict\ Ethical\ Improvement})$$

The designed system produces strictly less harm than an unconstrained baseline for every stakeholder.

4.2.5 Resultant Arguments from the State Transition Harm Model

The following theorem and propositions translate the required properties into more formal guarantees. The intuition being that if the only route to a forbidden state passes through a transition that is blocked, no execution can get to that state regardless of policy controls.

Theorem 4.1 (Blocking Propagation). *If every path from Q_0 to any $f \in F$ passes through some state q_b , and all transitions entering q_b are blocked by technically enforced controls, then $\text{Reach}_T(\mathcal{M}) \cap F = \emptyset$.*

Proof. Suppose $f \in \text{Reach}_T(\mathcal{M}) \cap F$. Then a path exists from Q_0 to f using only unblocked transitions. By hypothesis this path passes through q_b , requiring an unblocked transition to q_b . This is a contradiction to the assumption that all transitions entering q_b are technically blocked. $\square$

Proposition 4.2 (Monotonicity Implies Safety). *If $\mathcal{M}$ satisfies protection monotonicity, and $Q_0 \subseteq \{q \in \mathcal{Q}^* : \forall p \in \mathcal{P}^+, q(p) = 1\}$, then no reachable state has strictly fewer protective properties than the initial state, and any forbidden state characterised by the absence of protective properties is unreachable.*

Proof. By protection monotonicity every reachable state q' satisfies $q_0 \sqsubseteq q'$ for its initial state q_0 . Since q_0 holds all protective properties by assumption, q' must also hold all protective properties. Any forbidden state $f \in F$ defined by the absence of at least one protective property therefore cannot be reached. $\square$

Corollary 4.3 (Exact Harm Bound Under Exclusivity). *If $\mathcal{M}$ satisfies state exclusivity, then $\mathcal{H}(\mathcal{M})$ exactly characterises worst-case system harm.*

4.3 Ethical Dependency Graph

4.3.1 Decision Structure

Definition 4.17 (Lifecycle Stages). A totally ordered finite set $\Phi = \{\phi_1 < \phi_2 < \dots < \phi_k\}$ of research lifecycle stages, declared by the researcher. It's ordering represents the temporal precedence. A decision at stage ϕ_a must be made before any decision at stage ϕ_b where $\phi_a < \phi_b$. For this project, the lifecycle stages are defined as:

$$\Phi = \{\phi_s, \phi_p, \phi_i, \phi_o, \phi_a, \phi_b, \phi_r\}$$

Denoting scoping, planning, implementation, operation, analysis, publication and retention respectively.

Declaring stages during planning prevents ethical justification from being constructed retrospectively. A decision cannot borrow legitimacy from something that had not yet been determined when it was made.

Definition 4.18 (Edge Types). A finite set T of dependency types, which is declared explicitly during the planning phase to reflect the kinds of ethical relationships present in the research. Within this project, we define three edge types.

$$T = \{\tau_e, \tau_d, \tau_c\}$$

Where each edge is either enabling, discharging or constraining. Such that:

$$d_i \xrightarrow{\tau_e} d_j \implies d_j \text{ is ethically permissible only if } d_i \text{ holds}$$

$$d_i \xrightarrow{\tau_d} d_j \implies d_i \text{ absorbs the exposure introduced by } d_j$$

$$d_i \xrightarrow{\tau_c} d_j \implies d_i \text{ forecloses the decision space otherwise be available to } d_j$$

In each case, the ethical force of d_j is dependant on d_i remaining in place. The reversal of d_i would destabilise that relationship.

The three edge types capture distinct ethical relationships between decisions. An enabling edge means one decision is only permissible because another holds. A discharging edge means one decision absorbs the exposure introduced by another. A constraining edge means one decision narrows the options available to another. Together they allow the graph to represent not just what was decided, but why each decision was legitimate.

Definition 4.19 (Decision). A tuple $d = (\iota, \phi, E(d), M(d), \mathcal{A}(d))$ where ι is a unique identifier, $\phi \in \Phi$ is the lifecycle stage, $E(d) \in \mathcal{K}$ is the exposure vector, $M(d) \in \mathcal{K}$ is the mitigation vector, and $\mathcal{A}(d) \subseteq \mathcal{C}$ is the set of controls enforcing the decision.

A decision is not simply a design choice. It carries a formal record of its exposure, mitigation, and enforcing controls, making the graph a justificatory record rather than a dependency map.

4.3.2 Graph Structure

Definition 4.20 (Ethical Dependency Graph). A tuple:

$$\mathcal{G} = (D, E_{\text{dep}}, t, E(\cdot), M(\cdot), \mathcal{A}(\cdot))$$

where D is a finite set of research decisions and $E_{\text{dep}} \subseteq D \times D$ is the dependency relation of type $t : E_{\text{dep}} \rightarrow T$.

Written $d_i \xrightarrow{\tau} d_j$ to mean $(d_i, d_j) \in E_{\text{dep}}$ of type $t(d_i, d_j) = \tau$

The graph is the structure that holds the entire justificatory record together. Decisions are the nodes, dependencies are the edges, and the edge types record the nature of each relationship. A denser graph, with more dependency edges, is generally preferable. It means fewer decisions are self-supporting and more of the ethical burden is distributed across technically enforced relationships. A graph that satisfies the required properties is a proof that the reasoning behind the system is coherent.

Definition 4.21 (Root Node). A decision $d \in D$ is a root node *iff* it has no incoming edges in $\mathcal{G}$. Root nodes are the foundational commitments of the framework. They require the strongest justifications, and have the largest effects if reversed.

Root nodes are the foundational commitments which all other decisions derive their permissibility. Because nothing precedes them, they bear the greatest justificatory burden and carry the highest reversal cost.

Definition 4.22 (Ancestor Set). The ancestor set of d_j is:

$$\text{anc}(d_j) = \{d_i \in D : \exists \text{ directed path from } d_i \text{ to } d_j \text{ in } \mathcal{G}\}$$

The ancestor set of a decision is the full chain of prior commitments that together establish its permissibility. A decision with no technically enforced ancestors in its chain is a gap in the ethical record.

Definition 4.23 (Cascade Set). The cascade set of d_i is the set of decisions invalidated by its reversal through any enabling edges.

$$\text{cas}(d_i) = \{d_j \in D : d_i \in \text{anc}(d_j) \wedge \text{every path from } d_i \text{ to } d_j \text{ uses only } \tau_e \text{ edges}\}$$

The cascade set captures the downstream consequences of reversing a decision. A decision with a large cascade set is load-bearing. Removing invalidates both itself and any dependant decision that is connected via an enabling edge.

4.3.3 Exposure and Mitigation

Definition 4.24 (Cumulative Exposure). Defined recursively over the ancestor set:

$$\hat{E}(d_j) = E(d_j) \oplus \bigoplus_{d_i \in \text{anc}(d_j)} E(d_i) \ominus \bigoplus_{d_k \in \text{anc}(d_j)} M(d_k)$$

where $\oplus$ is componentwise maximum and $\ominus$ is componentwise subtraction:

$$(\kappa_1 \ominus \kappa_2)(s) = \max(0, \kappa_1(s) - \kappa_2(s)) \quad \forall s \in \mathcal{S}$$

Cumulative exposure tracks the net harm that remains after accounting for every mitigation in a decision's ancestry. It is the running balance of the ethical debt that a decision carries forward through the graph.

Definition 4.25 (Vulnerability). A decision d_j is vulnerable with respect to stakeholder $s \in \mathcal{S}$ iff it has a positive cumulative exposure for s and none of its ancestors are technically enforced:

$$\hat{E}(d_j)(s) > 0 \wedge \nexists d_k \in \text{anc}(d_j) : M(d_k)(s) > 0 \wedge \mathcal{A}(d_k) \cap \mathcal{C}_T \neq \emptyset$$

The vulnerability set of $\mathcal{G}$ is $\mathcal{V}(\mathcal{G}) = \{(d_j, s) : d_j \text{ is vulnerable w.r.t. } s\}$

A vulnerable decision is one that carries positive cumulative exposure without any technically enforced mitigation in its ancestry. It represents a gap where harm has been introduced but left unaddressed by the architecture. The vulnerability set collects all such gaps across the entire graph.

Definition 4.26 (Reversal Cost). The cost of reversing decision d_i defined as:

$$\rho(d_i) = |\text{cas}(d_i)| + \sum_{s \in \mathcal{S}} \sum_{d_j : d_i \xrightarrow{\tau_d} d_j} M(d_i)(s)$$

Reversal cost makes the load-bearing decisions explicitly. A high reversal cost means that removing a decision does not just reintroduce its own exposure but collapses the justification of all subsequent decisions.

4.3.4 Required Properties for the Ethical Dependency Graph

Once again, to meet the requirements laid out in previous sections, we require a set of properties to be valid in order for our needs to be met. Each of these respective properties implies their following statement.

$$\mathcal{V}(\mathcal{G}) = \emptyset \quad (\textbf{Full Soundness})$$

Every decision that introduces exposure for a stakeholder has a technically enforced mitigating ancestor. No stakeholder is left unprotected by design.

$$E_{\text{dep}} \text{ is acyclic} \quad (\textbf{Well-Foundness})$$

No decision is circularly dependent on itself or any of its descendants. This ensures that ethical justification flows forward without circular reasoning.

$$\forall (d_i, d_j) \in E_{\text{dep}}, \phi(d_i) \leq \phi(d_j) \quad (\textbf{Lifecycle Consistency})$$

No decision depends on a decision made at a later lifecycle stage. Ethical justification must flow forward through time, with later decisions depending only on earlier ones.

$$\forall d_j \notin \text{Roots}, \text{anc}(d_j) \neq \emptyset \quad (\textbf{No Orphans})$$

Every non-root decision has at least one explicit predecessor that establishes its permissibility. No decision may appear self-justifying without being declared a root node.

4.3.5 Resultant Arguments from the Ethical Dependency Graph

Proposition 4.4 (Lifecycle Consistency Implies Well-Foundness). *If $\mathcal{G}$ satisfies lifecycle consistency then E_{dep} is acyclic.*

Proof. A cycle would require $\phi(d_{i_1}) \leq \phi(d_{i_2}) \leq \dots \leq \phi(d_{i_1})$, forcing all decisions in the cycle to share the same lifecycle stage. Irreflexivity of the dependency relation therefore prohibits the cycle. $\square$

Proposition 4.5 (Vulnerability Detection Complexity). *$\mathcal{V}(\mathcal{G})$ can be computed in $O(|D|^2 \cdot |\mathcal{S}|)$ time.*

Proof Sketch. Since $\mathcal{G}$ is a directed acyclic graph (DAG) by well-foundness, a topological sort exists and can be computed in $O(|D| + |E_{\text{dep}}|)$. For each node in topological order, $\hat{E}(d_j)$ is computed from already-computed ancestor values in $O(|\text{anc}(d_j)| \cdot |\mathcal{S}|)$. Summing over all nodes gives $O(|D|^2 \cdot |\mathcal{S}|)$ in the worst case. $\square$

4.4 Joint Claims

When both frameworks are constructed over the same system, the following claims can be jointly asserted.

1. The system is structurally incapable of reaching forbidden states, and that incapability is grounded in coherent and technically enforced reasoning.
2. Every technically enforced control discharges a specific identified ethical obligation, and every ethical obligation has a corresponding technical enforcement.
3. The impact of reversing any design commitment is calculable in terms of which states become reachable, and which downstream decisions are invalidated (and in extension how much exposure is re-introduced).
4. Under state exclusivity, $\mathcal{H}(\mathcal{M})$ exactly characterises worst-case system harm with no approximation error, and $\Delta\Gamma$ is a comparison of actual systems rather than abstract models.

4.5 Weaknesses

Both frameworks share a fundamental limitation in the fact that they are only as accurate as the values assigned to their inputs. Harm vectors and exposure mitigation vectors are assigned by the researcher. This means the formal guarantees produced by both frameworks are contingent on the honesty and accuracy of those arguments, which the frameworks themselves cannot verify. A researcher who assigns low harm values to genuinely dangerous transitions produces a formally compliant system that provides false assurances. This is a subtler version of the same procedural ethics problem identified in the literature review. However, the inherent act of documenting and producing an ethically proven system allows for peers and fellow researchers to hold ethical decisions to account.

The STHM also has several additional weaknesses. Governance states are difficult to verify empirically, and may degrade over time. A system, that begins in an audited state may drift toward a more restricted state as access controls accumulate technical debt and monitoring is deprioritised. Governance monotonicity is therefore an assumption and requires ongoing verification, which reintroduces a human compliance dependency that the model was designed to avoid. Liveness follows a similar problem, as destruction depends on time-triggered processes and institutional procedures that exist outside the system boundary, making it closer to a policy commitment that technically enforceable. However, by intentionally limiting the time-frame of a project, many of these risks can be mitigated.

The EDG also has its own weaknesses. Specifically, the granularity of a decision remains unspecified. Decomposing a single decision into finer sub-decisions produces a denser graph with different soundness properties, meaning the framework’s outputs are sensitive to how decisions are defined. A related issue is that full soundness does not require mitigation strength to be proportionate to exposure strength. A decision with high exposure can therefore be formally sound against a weak mitigation, leaving residual harm unaddressed. The EDG also cannot detect conflicts between decisions that share no dependency edge, meaning a valid graph could contain decisions that are individually sound but mutually inconsistent.

There is one more residual limitation worth stating clearly. State exclusivity is a constraint on a system’s design rather than on the underlying hardware or operating system. At the level of CPU caches or network buffers, data will inevitably exist in multiple physical locations simultaneously. State exclusivity therefore operates at the logical level of the research system rather than as a physical guarantee. The axiom is genuinely enforceable for the purposes of the formal model, but a determined adversary with physical access to the hardware could potentially observe data in states that the model asserts it cannot occupy. This does not undermine the framework’s value, it simply means that state exclusivity

resolves the multiple logical state problem at the level of abstraction at which a program runs. This is realistically the only level at which the risk can be practically reasoned about. This observation supports the argument that physical security is a necessary corollary to the formal guarantees offered here and should be maintained as an additional policy-based constraint. Beyond this, a well-formed program operating at a sufficiently low level (See 6.5.3) may be able to mitigate certain in-memory attacks by handling data only in small discrete units, for example by operating on a single HTTP frame at a time. This approach reduces the window during which potentially dangerous data resides in memory, making such attacks more difficult to execute.

4.6 Closings on Verifiability

Formal verification changes what can be claimed. Rather than asserting that a system handles data responsibly, a researcher can demonstrate that certain harms are structurally unreachable, and that design decisions are well-justified. These are stronger claims than compliance-based frameworks can produce, and they are claims that can be scrutinised.

This matters particularly in Tor research because the stakes of failure are high and often irreversible. Exposure to traumatic content or the deanonymisation of a stakeholder are actions that cannot be undone after the fact. Grounding safety properties in the architecture of the system itself provides guarantees that researcher goodwill and institutional oversight alone cannot.

This framework is not presented as a complete solution. The weaknesses outlined above are valid, and the formal guarantees are dependent on honest and accurate input assignments. By converting ethical claims into proofs, reasoning can be measured against a standard baseline, and reviewed externally. That is what substantive ethics requires, and what the existing literature has so far failed to provide.

The natural question at this point is why this complexity is necessary at all. Existing ethical instruments are well-established, but their guarantees are bound by temporality and the researcher’s intentions. What they do not address is what a deployed system is structurally capable of doing, which is exactly the point at which harm can occur without any individual acting in bad faith.

The formal overhead of the STHM and EDG is therefore the price of making claims that are precise and honest about their scope. Ethical guarantees that rest on the architecture of the system itself are qualitatively different from those that rest on the researcher, and that difference is what makes them verifiable.

Chapter 5

Ethical Verification in Practice

The frameworks presented in the previous chapter are only meaningful if they can be instantiated against a real research system. We therefore develop two crawler implementations over a shared formal vocabulary, allowing their ethical and performance properties to be directly compared.

5.1 System Descriptions

The Naive Crawler ($\mathcal{M}_{\text{naive}}$)

The naive crawler is a simple, python-based implementation with no protective design. Python was chosen for its loose memory control and similarities to projects found in (Bergman & Popov, 2023; De Pascale et al., 2024; Kuhn et al., 2024; Pastor-Galindo et al., 2024). It crawls all URLs it discovers on a site without checking the content type first. It makes no attempt to conceal its identity and issues standard HTTP requests with a default user agent. All raw page content is fetched and written directly to a database or file store with no filtering or minimisation at any stage. The researcher interacts directly with the stored content. The system has no audit logging, encryption, retention policy, or incident procedures. No URL hashing or anonymisation of any kind is performed.

The Restricted Crawler ($\mathcal{M}_{\text{restricted}}$)

The restricted crawler is written in a lower level language with fine-grain memory control. It should use a secure allocator (see 6.5.2), which zeroes memory on deallocation and may have additional security-focused features. This prevents residual content from lingering in memory after a buffer is freed. Before any URL is fetched, an HTTP HEAD request is issued to determine the content type. Non-textual or unexpected content types are rejected before any payload is then downloaded. The system should run disklessly via netboot on a physically isolated and airgapped machine. The system should be hardened to the standard of at least that of CIS Level 2 benchmarks. The institution controls that machine, the researcher does not. The database sits on a separate machine in a network-isolated VLAN. The database itself should be explicitly append-only. Nothing written can be modified or deleted. Full disk encryption is enforced on all storage. Crawled URLs are stored only in hashed form so that the database cannot serve as a directory of discovered services. Researchers never access raw page content or transient memory. All potentially harmful material is filtered and purged before any human interaction. Only aggregate performance metrics and anonymised statistics can be published.

5.2 Shared Formal Vocabulary

Before applying either model, we must declare the project-specific constants required by both frameworks.

Stakeholder Set $\mathcal{S}$

$$\mathcal{S} = \{s_P, s_I, s_R, s_N, s_X\}$$

which denotes direct Tor participants, indirectly represented individuals, researchers, institutions and external actors respectively.

Property Set $\mathcal{P}$

Each data artefact is described by the following observable binary properties:

Symbol	Property
p_{enc}	The artefact is encrypted at rest
p_{anon}	The artefact contains no raw URL or service identifier
p_{filt}	The artefact been filtered and contains no raw page content
p_{min}	The artefact contains only minimised, derived output with no raw HTML, headers, or media
p_{inst}	The artefact resides on institutionally controlled infrastructure
p_{aud}	All access to the artefact is logged in a Merkle-tree-protected audit trail, cross-referenceable against physical entry sheets
p_{raw}	The artefact contains raw page content
p_{human}	The artefact is directly visible to a human researcher without filtering

Degeneracy Axioms (Δ)

The following property combinations are logically incompatible and are excluded from the admissible state space $\mathcal{Q}^*$.

$$p_{\text{filt}} = 1 \implies p_{\text{raw}} = 0$$

$$p_{\text{min}} = 1 \implies p_{\text{raw}} = 0$$

A filtered or minimised artefact cannot simultaneously contain raw content.

Protective Properties ($\mathcal{P}^+$)

$$\mathcal{P}^+ = \{p_{\text{enc}}, p_{\text{anon}}, p_{\text{filt}}, p_{\text{min}}, p_{\text{inst}}, p_{\text{aud}}\}$$

Forbidden States (F)

The following states must never be reached:

ID	Characterisation
f_1	$p_{\text{raw}} = 1 \wedge p_{\text{human}} = 1$: raw content is viewed by a researcher
f_2	$p_{\text{raw}} = 1 \wedge p_{\text{inst}} = 0$: raw content exists outside institutional infrastructure
f_3	$p_{\text{anon}} = 0 \wedge p_{\text{human}} = 1$: unhashed URLs are visible to a researcher
f_4	$p_{\text{raw}} = 1 \wedge p_{\text{enc}} = 0$: any raw data is stored unencrypted at rest

Trigger Set (Λ)

$$\Lambda = \{\lambda_{\text{recv}}, \lambda_{\text{buf}}, \lambda_{\text{write}}, \lambda_{\text{xform}}, \lambda_{\text{view}}, \lambda_{\text{pub}}, \lambda_{\text{del}}, \lambda_{\text{leak}}, \lambda_{\text{breach}}\}$$

These triggers denote receive from network, buffer in memory, write to persistent storage, transform, expose to human view, publish, delete, unintentional leak, and security breach respectively. The final two are involuntary and cannot be prevented by policy alone.

Control Set ($\mathcal{C}$)

ID	Description	Type
c_1	Residual content cannot persist in memory beyond its processing scope	$\mathcal{C}_T$
c_2	Non-textual content is rejected before any payload is retrieved	$\mathcal{C}_T$
c_3	Persistent storage is append-only	$\mathcal{C}_T$
c_4	All persistent identifiers are cryptographically hashed before storage	$\mathcal{C}_T$
c_5	All persistent storage is encrypted at rest	$\mathcal{C}_T$
c_6	The crawling environment is physically and network isolated from external systems	$\mathcal{C}_T$
c_7	The crawling machine has no persistent local storage	$\mathcal{C}_T$
c_8	The implementation provides memory safety guarantees such that residual content cannot persist through unsafe memory access.	$\mathcal{C}_T$
c_9	The extraction stage is architecturally incapable of accumulating raw page content	$\mathcal{C}_T$
c_{10}	Raw and unhashed data is inaccessible to researchers by architectural construction	$\mathcal{C}_T$
c_{11}	All machines operate under institutional jurisdiction and must remain within a physically controlled on-premise environment	$\mathcal{C}_T$
c_{12}	The operating environment is declaratively built and verified against a published hardening standard at every image build	$\mathcal{C}_T$
c_{13}	All log entries and tracing spans are protected by a Merkle-tree integrity structure	$\mathcal{C}_T$
c_{14}	Researcher access to the secure environment is recorded on physical entry sheets maintained independently of the digital infrastructure	$\mathcal{C}_P$
c_{15}	Researchers sign and adhere to a documented code of conduct	$\mathcal{C}_P$
c_{16}	Research operates within a scope approved by an ethics review body	$\mathcal{C}_P$
c_{17}	Incidents reported via a documented reporting procedure through a documented reporting procedure	$\mathcal{C}_P$

Under zero-trust, only $\mathcal{C}_T$ controls c_1 through c_{13} are treated as reliable. Controls c_{14} through c_{17} are necessary complements to the technical architecture but cannot serve as sole safeguards, as each depends on human compliance rather than architectural enforcement.

Audit logs are themselves a tampering target. An adversary with sufficient access could modify or delete log entries after the fact, which would undermine the auditability the framework depends on. Two controls address this together. First, all log entries and tracing spans are written into a Merkle tree (c_{13}). Any modification to any entry invalidates the root hash and is cryptographically detectable. Second, a physical entry sheet (c_{14}) records each occasion a researcher enters or exits the secure environment. These sheets are kept independently of the digital infrastructure and can be cross-referenced against the Merkle-tree log at any later point. The Merkle tree covers digital tampering. The entry sheets cover the case where the logging system itself is compromised.

Lifecycle Stages (Φ)

$$\Phi = \{\phi_s < \phi_p < \phi_i < \phi_o < \phi_a < \phi_b < \phi_r\}$$

denoting scoping, planning, implementation, operation, analysis, publication, and retention/destruction respectively.

5.3 State Transition Harm Model

Initial States

Naive Crawler (Q_0^{naive})

Data enters in state q_0 where all protective properties are absent:

$$q_0 = \{p_{\text{enc}} = 0, p_{\text{anon}} = 0, p_{\text{filt}} = 0, p_{\text{min}} = 0, p_{\text{inst}} = 0, p_{\text{aud}} = 0, p_{\text{raw}} = 1, p_{\text{human}} = 0\}$$

Restricted Crawler ($Q_0^{\text{restricted}}$)

Data enters in the following state at the network boundary, immediately after the HEAD pre-check has passed and before the GET response body is consumed:

$$q_\star = \{p_{\text{enc}} = 0, p_{\text{anon}} = 0, p_{\text{filt}} = 1, p_{\text{min}} = 0, p_{\text{inst}} = 1, p_{\text{aud}} = 1, p_{\text{raw}} = 1, p_{\text{human}} = 0\}$$

Data enters the restricted crawler at the network boundary after the HEAD pre-check and before the response is processed. At this point, the content type has already been confirmed, so $p_{\text{filt}} = 1$ holds on ingest. Given the response body stream is unprocessed, the data at this stage is considered as $p_{\text{raw}} = 1$. We can also assert that $p_{\text{anon}} = 0$ as hashing occurs at extraction rather than when the page is fetched. The machine and logging infrastructure being in place before any data is received ensures that p_{inst} and p_{aud} hold. Controls c_1 , c_8 , and c_9 together ensure that raw content cannot persist in memory beyond the extraction window.

Naive Crawler Transitions

The following tables enumerate the ethically significant transitions for each system with their harm vectors and assigned controls. Harm is expressed as a vector over $\mathcal{S} = \{s_P, s_I, s_R, s_N, s_X\}$ using severity values $\{0, 1, 2, 3\}$, where each position corresponds to the stakeholder classes defined above in the same order. A value of 0 indicates no harm to that class and 3 indicates the highest severity. Each row represents a single transition, identifying the trigger that causes it, the harm it introduces if unblocked, and whether the controls assigned to it are technically enforced.

Transition	Trigger	$\kappa (s_P, s_I, s_R, s_N, s_X)$	Controls $\mathcal{A}$	Blocked?
e_1	λ_{recv}	$(2, 1, 1, 1, 0)$	—	No
e_2	λ_{buf}	$(2, 1, 1, 1, 0)$	—	No
e_3	λ_{write}	$(3, 2, 2, 2, 1)$	—	No
e_4	λ_{view}	$(3, 3, 3, 3, 2)$	c_{15}	No ($\mathcal{C}_P$ only)
e_5	λ_{pub}	$(3, 3, 2, 3, 3)$	c_{15}	No ($\mathcal{C}_P$ only)
e_6	λ_{leak}	$(3, 3, 3, 3, 3)$	—	No

Table 5.1: Transitions for $\mathcal{M}_{\text{naive}}$

Transitions e_3 and e_4 reach forbidden states f_4 and f_1 respectively. No technically enforced control blocks either of them. The naive system therefore violates Zero-Trust Safety. The buffering transition e_2 is included to reflect that raw content is held in memory with no zeroing guarantee, leaving it exposed between receipt and write.

Restricted Crawler Transitions

Transition	Trigger	$\kappa (s_P, s_I, s_R, s_N, s_X)$	Controls $\mathcal{A}$	Blocked?
e_7	λ_{recv}	$(1, 0, 0, 0, 0)$	c_2, c_8, c_1	Partial: c_2 rejects non-textual content; c_8 and c_1 bound residual content to the processing window
e_8	λ_{buf}	$(1, 0, 0, 0, 0)$	c_9, c_1	Partial: c_9 bounds the buffer to href values only and c_1 zeroes memory on deallocation
e_9	λ_{xform}	$(0, 0, 0, 0, 0)$	c_9, c_4	Permitted (harm-free): c_4 hashes all identifiers, acquiring $p_{\text{anon}} = 1$ and $p_{\text{min}} = 1$
e_{10}	λ_{write}	$(0, 0, 0, 0, 0)$	c_3, c_5, c_{11}	Permitted (harm-free): only hashed records reach storage, encrypted at rest under append-only semantics
e_{11}	λ_{view}	$(0, 0, 0, 0, 0)$	c_{10}, c_{13}, c_{14}	Permitted (harm-free): researcher access is to derived outputs only, logged and corroborated by the entry sheet
e_{12}	λ_{view}	$(3, 3, 3, 3, 2)$	c_7, c_9, c_{10}, c_{11}	Yes ($\mathcal{C}_T$): raw content reaching a researcher is blocked by diskless operation, the streaming extractor, architectural inaccessibility, and institutional jurisdiction
e_{13}	λ_{leak}	$(1, 0, 1, 1, 0)$	c_6, c_{12}	Substantially blocked by network isolation and a hardened OS
e_{14}	λ_{del}	$(0, 0, 0, 0, 0)$	c_{13}, c_{14}	Permitted (harm-free): destruction is recorded in the Merkle log and corroborated by the entry sheet

e_{12} is the critical blocked transition. Accumulation of raw content is prevented by c_9 . Local persistence is prevented by c_7 . Researcher access is prevented by c_{10} . Each prerequisite for a forbidden state is blocked by a technically enforced control and there is no path to any forbidden state. The restricted system therefore satisfies Zero-Trust Safety.

Verification of Required Properties

Zero-Trust Safety: $\text{Reach}_T(\mathcal{M}_{\text{restricted}}) \cap F = \emptyset$

Each forbidden state is blocked by technically enforced controls:

- f_1 requires $p_{\text{raw}} = 1 \wedge p_{\text{human}} = 1$. c_9 ensures raw content is never accumulated beyond the extraction window. c_7 ensures no local storage exists on which it could persist. c_{10} ensures no researcher-accessible interface exists. Together they enforce that no state satisfying f_1 is reachable.
- f_2 requires $p_{\text{raw}} = 1 \wedge p_{\text{inst}} = 0$. c_6 and c_{11} enforce that the crawling environment is both network-isolated and operating under institutional jurisdiction, preventing any artefact from residing outside that boundary.
- f_3 requires $p_{\text{anon}} = 0 \wedge p_{\text{human}} = 1$. c_4 hashes all identifiers before any write, acquired at e_9 . No raw URL or host identifier survives into any human-accessible state.
- f_4 requires $p_{\text{raw}} = 1 \wedge p_{\text{enc}} = 0$. c_9 ensures raw content is never written. c_5 encrypts all writes.

Protection Monotonicity

Every transition in $\mathcal{M}_{\text{restricted}}$ satisfies $q \sqsubseteq q'$. At e_9 , p_{anon} and p_{min} are acquired, both members of $\mathcal{P}^+$, while p_{raw} is cleared. As $p_{\text{raw}} \notin \mathcal{P}^+$, clearing it does not violate the ordering. p_{filt} is held from q_* and is never cleared. The protective property set is therefore monotonically non-decreasing along all permitted transitions.

Liveness

Every reachable state has a path to q_ω via λ_{del} , retriggered by the post-research destruction schedule. The append-only database (c_3) ensures no content can be silently removed, so all destruction events are auditable.

Ethical Cost and Improvement

The ethical cost of a system is computed over the set of unblockable permitted transitions E_P using the componentwise maximum of harm vectors.

For the naive crawler $\mathcal{M}_{\text{naive}}$, the dominant transitions are e_3 and e_4 . Both are unblocked and reach forbidden states. The worst-case harm vector is:

$$\Gamma(\mathcal{M}_{\text{naive}}) = (3, 3, 3, 3, 2)$$

For the restricted crawler $\mathcal{M}_{\text{restricted}}$, the extraction transition e_9 acquires $p_{\text{anon}} = 1$ and $p_{\text{min}} = 1$ such that all subsequent permitted transitions carry a harm vector of $(0, 0, 0, 0, 0)$. No raw content and no unhashed identifier survives into any human-accessible state. The only residual harm is from the receive and buffer transitions e_7 and e_8 , both of which are substantially mitigated by c_1 , c_8 , and c_9 . In the worst case:

$$\Gamma(\mathcal{M}_{\text{restricted}}) = (1, 0, 0, 0, 0)$$

Strict Ethical Improvement

$$\Delta\Gamma = \Gamma(\mathcal{M}_{\text{naive}}) - \Gamma(\mathcal{M}_{\text{restricted}}) = (2, 3, 3, 3, 2) \gg \mathbf{0}$$

The improvement is strictly positive across every stakeholder dimension. This satisfies strict ethical improvement and provides the quantitative basis for claiming that the design constraints have produced a meaningful difference in harm exposure.

5.4 Ethical Dependency Graph

The EDG records the reasoning behind the restricted crawler’s design and makes it auditable. Each decision is traced to the exposure it creates, the mitigation provided, and the controls that enforce it. Unlike the STHM, which proves structural safety, the EDG proves that the design decisions which produce that safety were deliberately reasoned and grounded in a coherent justificatory chain.

Decision Set

Each decision is represented as $d = (\iota, \phi, E(d), M(d), \mathcal{A}(d))$, where exposure and mitigation vectors are over $\mathcal{S}$.

The full decision set $D = \{d_1, \dots, d_{22}\}$ with exposure vectors, mitigation vectors, and assigned controls is given in Table A.1.

The key dependency edges of $\mathcal{G}$, with their types and interpretations, are enumerated in Table A.2.

The graph $\mathcal{G}$ is constructed over $D = \{d_1, \dots, d_{22}\}$. Unlike a single-rooted tree, $\mathcal{G}$ has three root nodes: d_1 , d_2 , and d_3 . Each represents a foundational commitment made at the scoping stage, before any implementation decisions were taken. d_1 is the decision to conduct empirical Tor research and is the origin of all ethical exposure in the graph. d_2 is the decision to use Rust and the Arti library, motivated by Tor’s tier-one Rust support and the memory safety guarantees the language provides. d_3 is the decision to avoid browser emulation, motivated by the architectural overhead it would introduce and the fact that

Tor’s network model renders most of its benefits irrelevant. Both d_2 and d_3 were known design commitments at the point d_1 was formally made, and their ethical properties were recognised rather than designed after the fact. The planning-phase decisions d_4 through d_8 express design intent without yet specifying mechanism. They record what the system must achieve ethically before any implementation choice is made about how to achieve it. The implementation-phase decisions d_9 through d_{17} then resolve each planning intent into a specific architectural commitment.

Verification of Required Properties

Full Soundness ($\mathcal{V}(\mathcal{G}) = \emptyset$)

d_1 is the sole source of positive exposure, with vector $(3, 2, 2, 2, 1)$. d_2 and d_3 carry zero exposure vectors. The cumulative mitigation delivered through the descendant decisions of d_1 must satisfy:

$$\hat{M}(d_j) \geq E(d_1) \text{ for all leaf decisions in the graph}$$

The five planning-phase decisions d_4 through d_8 each address a distinct dimension of d_1 ’s exposure, and each is resolved into at least one technically enforced implementation decision. The technically enforced controls c_1 through c_{13} together cover all five stakeholder dimensions. No leaf decision in $\mathcal{G}$ carries positive cumulative exposure without a technically enforced mitigating ancestor in $\text{anc}(d_j)$. The vulnerability set is therefore $\mathcal{V}(\mathcal{G}) = \emptyset$.

Well-Foundness

E_{dep} is acyclic. By Proposition 4.4, lifecycle consistency implies acyclicity. No edge in the graph points backward through Φ , so lifecycle consistency holds by construction. The cross-root edges $d_2 \rightarrow d_{11}$ and $d_3 \rightarrow d_{11}$ both run from ϕ_s to ϕ_i and do not introduce cycles.

Lifecycle Consistency

All edges in E_{dep} respect the ordering $\phi_s < \phi_p < \phi_i < \phi_o < \phi_a < \phi_b < \phi_r$. The binding cases are $d_{18} \rightarrow d_{22}$ and $d_{19} \rightarrow d_{22}$, where audit logging and entry sheet maintenance sit at ϕ_o and verified destruction sits at ϕ_r .

No Orphans

Every non-root decision has at least one root node in its ancestor set, reachable either directly or through intermediate nodes. The planning-phase decisions d_4 through d_8 are direct descendants of d_1 . Every implementation-phase decision is reachable from at least one root through the planning layer. No decision is self-justifying.

Reversal Cost Analysis

The reversal cost $\rho(d_i)$ determines how much harm is re-introduced and how many decisions are invalidated when a given design choice is removed. It identifies which decisions are most load-bearing. The full analysis is given in Table A.3.

Reversing d_7 produces the largest cascade because researcher isolation is the planning-phase obligation from which diskless operation, air-gapping, and publication restrictions all derive their permissibility. Reversing d_{12} produces the highest re-identification risk, as the database transitions from a set of opaque digests to a linkable record of discovered services. Reversing d_{11} produces no cascade but immediately threatens two forbidden states, since no other control prevents raw content from accumulating once the streaming extractor is replaced.

5.5 Joint Claims

Joint Claim 1 is satisfied. $\mathcal{M}_{\text{restricted}}$ is structurally incapable of reaching any state in F .

Joint Claim 2 is satisfied. Each technically enforced control (c_1 through c_{13}) corresponds to a specific discharging or constraining edge in $\mathcal{G}$. Each identified exposure vector has a technically enforced mitigating decision in its ancestor set. The Merkle-tree log (c_{13}) closes the audit-integrity gap that would otherwise leave the tamper-evidence claim dependent on policy controls alone.

Joint Claim 3 is satisfied. The reversal cost table (Table A.3) shows that the impact of removing any design commitment is measurable. The table identifies which forbidden states become reachable and which decisions therefore become invalid.

Joint Claim 4 is satisfied under state exclusivity. Data artefacts occupy exactly one state at any logical point in time via the various architectural constraints specified. This means $\mathcal{H}(\mathcal{M}_{\text{restricted}})$ is an exact characterisation of worst-case harm with no approximation from unmodelled concurrency. $\Delta\Gamma = (2, 3, 3, 3, 2)$ is a comparison of actual system harms rather than abstract models.

5.6 Summary

The two implementations show that the formal framework produces testable claims rather than ethical assumptions. The naive crawler fails Zero-Trust Safety because unblocked transitions lead directly to forbidden states. In contrast, the restricted crawler meets the required properties of the STHM. The EDG constructed therewith shows that every design choice contributing to that safety was deliberately reasoned.

The ethical improvement $\Delta\Gamma = (2, 3, 3, 3, 2)$ is non-trivial across all stakeholder dimensions. It provides the quantitative basis for the central claim of this dissertation. Rigorous ethical constraints can be embedded in Tor research systems as structural properties rather than procedural assertions. We further assert that these restrictions can measurably reduce the potential harm of an ethical system.

Visual summaries of the State Transition Harm Model and the Ethical Dependency Graph as applied to the restricted crawler are provided in Appendix B.1 and Appendix B.2.

Chapter 6

Implementation

This chapter describes the technical realisation of $\mathcal{M}_{\text{restricted}}$ as introduced in the practical use section above. It is organised around the same layers that the ethical framework addresses: the physical environment, the network, the operating system, and the software stack. For each layer, the relevant design decisions are described alongside their relationship to the formal framework established in Chapter 4.

We maintain the mindset that controls are not additions applied after a system was built, they are inherent to the architecture. This is necessary as it is the objective of this project to demonstrate that ethical rigour carries a calculable price rather than an abstract one.

6.1 Physical Environment and Access Controls

The crawler systems are housed in a locked room on institutional premises. Access is restricted to approved students, and is permitted only between 09:00 and 18:00 on weekdays. No remote shell access to the machines is available from outside the room, via both machine configuration and network policy.

This control follows directly from the zero-trust posture established in Section 3.1. An adversary with uncontrolled physical access may be able to access data in states that should be impossible. Treating physical isolation as a control expands the zero-trust boundary to the hardware layer, closing off a class of threats that no software control can address.

The limited access window is also relevant to the formal model. The λ_{view} trigger, which represents the exposure of data to human view, is only reachable while the room is occupied and the system is active. By bounding the hours during which this is possible, the set of execution paths that pass through λ_{view} transitions is limited. This reduces the reachable harm $\mathcal{H}(\mathcal{M})$ by removing paths that would otherwise be available in an always-on system.

6.2 Network Design and Segmentation

6.2.1 VLAN Separation

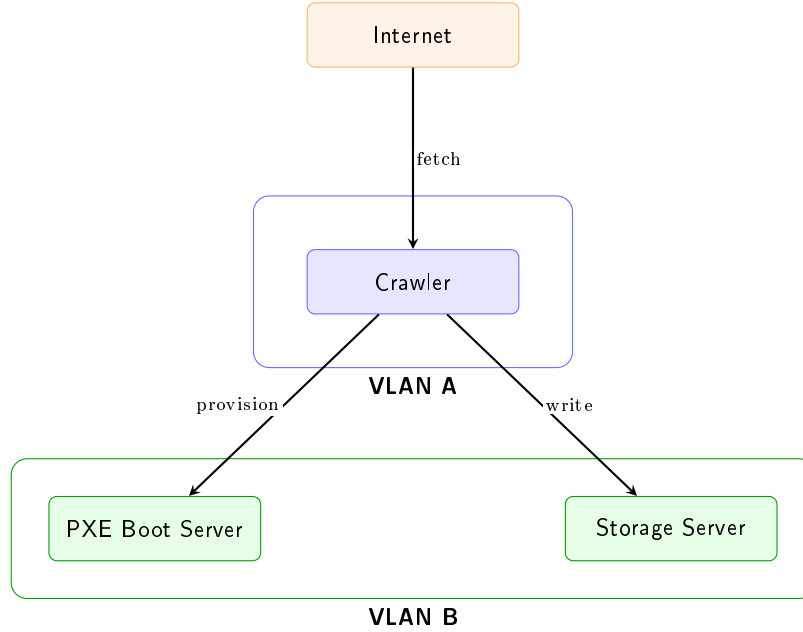


Figure 6.1: Network separation between VLAN A (crawler, outbound internet access) and VLAN B (air-gapped). Data flows unidirectionally from the internet through the crawler into VLAN B. No path returns from VLAN B to VLAN A or the internet.

The infrastructure is divided into two separate networks within the institutions network. Network A has outbound internet connectivity and is the environment in which the crawler itself operates. Network B is fully air-gapped from the internet and carries all persistent data, including the PXE boot server and the append-only data store. Both of these networks are blocked from making requests to any other machines on the institutional network.

This design directly enforces Protection Monotonicity (4.2.4). Once a data artefact moves from Network A to the governed state represented by Network B, there is no transition available that would return it to a less protected state. The transition from data in transit to data at rest therefore cannot introduce a path back toward an externally reachable state. The data store is structurally unreachable from any externally routable address, meaning that a compromise of Network A cannot propagate to the persistent dataset. This unidirectional constraint is enforced at the network layer, making it a technical ($\mathcal{C}_T$) control.

6.2.2 Air-Gapped PXE Boot Server

The PXE boot server, which provides the signed NixOS images (see 6.4.1) from which all crawling nodes are netbooted, resides on Network B and operates entirely offline. There is no path from the boot server to the internet, meaning the boot environment cannot be tampered with remotely. The PXE server itself boots from a signed USB image and mounts no persistent disks, making it fully ephemeral. Any modification to the boot image therefore requires physical presence with the signed boot drive, which cannot be replicated or accessed remotely.

This design forecloses a specific class of forbidden states. If the boot image could be modified remotely, it would be possible to introduce a configuration that weakens or removes other $\mathcal{C}_T$ controls without physical presence. The air-gapped boot server blocks all transitions that lead to that condition, ensuring that the set of technically enforced controls declared at planning time remains intact throughout operation.

6.3 Version Control and Repository Governance

All code, system configuration, and the paper itself are stored in a remote Git repositories hosted under a GitHub organisation. Repository access is restricted to approved personnel, who are added to the organisation explicitly. No repository is publicly accessible.

This is relevant to the EDG in the same way that declarative configuration is (see 6.4.1). The version-controlled record of every change to the codebase and NixOS configuration constitutes an auditable history of the decisions that shaped the running system. Additional protections were put in place to ensure that the git history could not be tampered with. This makes decisions made stated in the EDG constructable from this history.

This paper was also included in the same organisational group as the source code, which ensures that ethical reasoning and technical implementation share a common provenance. Changes to the ethical argument and changes to the system that argument describes are recorded in the same history. This reduces the risk of the two drifting apart in ways that would undermine the joint claims in 4.4.

6.4 Operating System Specifications

6.4.1 NixOS and Declarative Configuration

All systems in the research environment run NixOS, a Linux distribution whose defining property is that the system configuration is expressed declaratively. This specification covers packages, services, users, kernel parameters, and security settings. The result is that any

running NixOS image is, by construction, the exact output of its declaration. With the proper configuration, there is no mechanism by which a package can be installed, a service enabled, or a user created outside of that declaration.

One of the weaknesses of the EDG identified in the previous chapter is that governance states may degrade over time as controls accumulate technical debt. Declarative configuration directly addresses this, since the running state of the system is always identical to its declared specification, making governance monotonicity structurally enforced rather than assumed. The configuration files are retained as part of the project record and can be inspected to verify that the $\mathcal{C}$ controls were respected.

6.4.2 CIS Benchmark Hardening

All systems are hardened to the CIS Level 2 (CIS Center for Internet Security, 2017) benchmark for Linux, which is a set of secure configuration recommendations applied as a matter of institutional standard. CIS hardening is a $\mathcal{C}_T$ control in the sense of Definition 4.3. It is expressed in the declarative NixOS configuration and therefore cannot be overridden without modifying the signed image. This means it cannot be bypassed by policy failure alone, which is the condition under which $\text{Reach}_T(\mathcal{M})$ is computed. Forbidden states that depend on the absence of process isolation or unrestricted module loading are therefore unreachable under zero-trust assumptions.

6.4.3 Encrypted Storage

The storage server runs a signed, secure-booted NixOS image and operates with no externally exposed interfaces beyond those declared explicitly in its configuration. The PostgreSQL instance it hosts is configured in append-only mode, prohibiting deletions and modifications to stored records. Read access to the dataset is restricted to a small set of securely configured service accounts used exclusively for metrics retrieval.

The persistent data volume is encrypted using Linux Unified Key Setup (LUKS) and formatted with `btrfs`. Encryption keys are generated cryptographically using a secure random source. Each key is enrolled to the Trusted Platform Module (TPM) of the storage machine. This binding ensures that the encrypted volume can only be unsealed by the specific hardware on which it resides. No user passwords are configured on the machine, and no interactive login path exists. Physical removal of the storage drive therefore yields a volume that cannot be decrypted outside of the intended machine.

6.4.4 Secure Boot with a Custom Enrolled Key

All machines use UEFI Secure Boot with project-specific keys enrolled directly in the firmware. This applies to each booted environment in the system, covering the PXE server, the crawling nodes, and the persistent datastore. Each boots from an image signed with the project key, and the firmware will refuse to load any image that has not been signed by the research team. The keys themselves are not stored in any of the images, but in a central key store accessible only to the research team. This means that any new image needs to be signed centrally before it can be deployed, mitigating λ_{breach} state transitions that could arise from a substituted or modified image.

This control forms the final layer of the chain of trust from firmware to runtime. Disk encryption protects data at rest from physical removal of a drive. CIS hardening limits what a running system can do. Secure Boot with a custom key ensures that the system actually running is the one that was signed and declared. Without this layer, an attacker with physical access could substitute a modified image that removes certain security measures.

6.5 Software Stack

The crawler is implemented as a multi-stage asynchronous pipeline in Rust, structured around typed channels that enforce data boundaries between each processing stage. Figure 6.2 shows the overall structure and the network boundary each stage operates within.

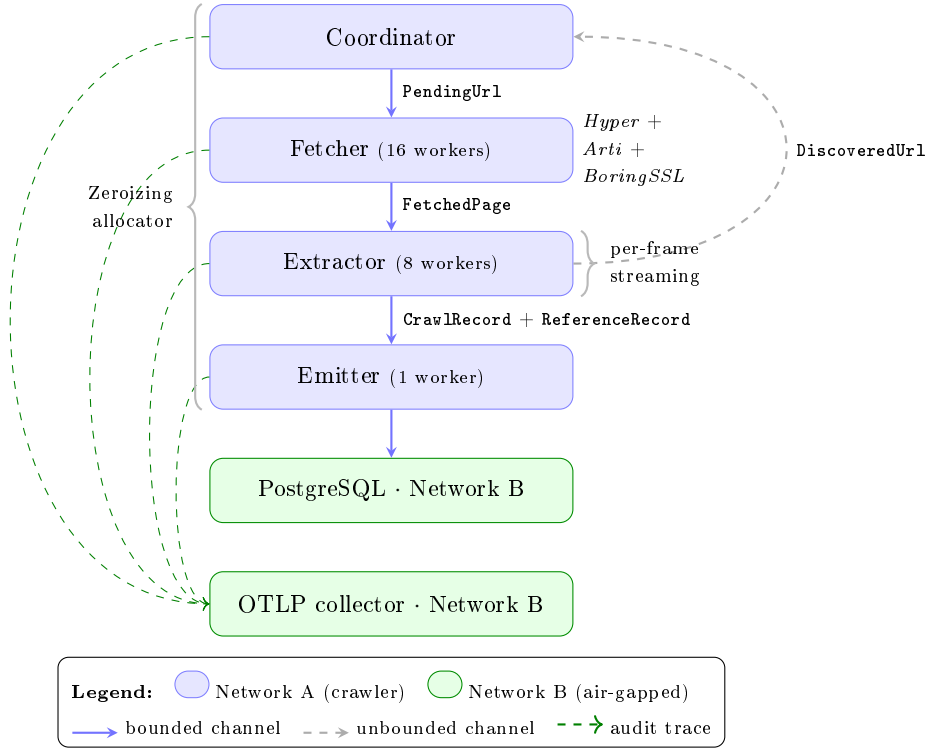


Figure 6.2: The crawler pipeline running on Network A passes data through typed channels to the air-gapped stores on Network B.

6.5.1 Rust

The crawler is implemented in Rust. The most relevant property of the language for this framework is memory safety. Rust’s ownership and borrowing model ensures, at compile time, that a class of memory errors common in C and C++ cannot occur in safe Rust code.

This matters for the formal model because memory corruption vulnerabilities represent a class of involuntary triggers. A hidden service that can induce a use-after-free in the crawler has effectively caused a λ_{breach} transition that was not declared in the trigger set and is therefore outside the modelled transition relation entirely. Rust ensures that the set of reachable states is confined to those reachable through the declared trigger set Λ .

Rust is also the language of the Tor Project’s own client library, Arti (See 6.5.6). The existence of a well-maintained official Rust Tor client was a motivating factor for the use of Rust, for the project as a whole.

6.5.2 Zeroing Allocator

The project uses a custom allocator that zeroes all memory regions on deallocation, ensuring that residual content cannot linger in memory long enough for data forensics to be practical. In the state transition model, a buffer that has been freed but not zeroed occupies an ambiguous position, which is the condition described in the weaknesses section, where state exclusivity operates at the logical level of the program rather than the physical level of hardware. The zero-allocator reduces this vulnerability. Once a state transition to a subsequent stage is complete, the buffer is freed and its content no longer exists in any physically readable form, enforcing state exclusivity as close to the hardware level as possible.

6.5.3 Per-Frame Processing and Immediate Discard

The crawler processes HTTP responses one frame at a time, rather than accumulating a full response body before processing. Each HTTP frame is handled as it arrives and the frame buffer is released as soon as processing of that frame is complete.

This decision has two immediate benefits. The first is a reduced memory footprint, since each frame is only needed at the time of consumption and can be processed asynchronously and independently of any other frame. The second follows from the interaction with the zero-allocation allocator described above. Because each frame buffer is freed and zeroed immediately after use, the window during which any given frame’s content exists in readable memory is bounded by the duration of its own processing.

6.5.4 TLS and HTTP/2 Fingerprint Spoofing

One of the mechanisms by which a hidden service operator can identify a research crawler is through the analysis of TLS ClientHello and HTTP/2 SETTINGS fingerprints. These fingerprints encode characteristic patterns in the TLS extension order, cipher suite preferences, and HTTP/2 frame configuration produced by a specific software stack. By default, TLS and HTTP/2 configurations produce fingerprints that differ from those of common browsers. This paper opted to apply patches to the underlying BoringSSL (Google, 2014) and Hyper (McArthur, 2022) libraries in order to produce fingerprints consistent with those of various Tor Browser versions. From the perspective of a hidden service receiving a request, the crawler is indistinguishable from an ordinary Tor Browser session. Other access patterns

may remain detectable, such as the absence of JavaScript execution¹. TLS and HTTP/2 fingerprint spoofing was therefore determined to provide sufficient cover for the purposes of this research.

This control is relevant to the formal model through the λ_{breach} and λ_{leak} triggers. If a hidden service operator identifies the crawler as a research tool, the research infrastructure may become a target. A successful attack on the infrastructure could force unintended transitions, including leaks of operational data or compromise of the systems holding the persistent dataset. In the EDG, this decision functions as a discharging edge. It absorbs the exposure introduced by the decision to operate at all, for the stakeholders who would be harmed if the research infrastructure were identified and attacked.

6.5.5 Hyper and HTTP Internals

The HTTP layer of the crawler is built on Hyper (McArthur, 2022), the de facto HTTP client implementation in the Rust ecosystem. Hyper handles the low-level mechanics of connection management, request serialisation, and response parsing. It was chosen both for its performance characteristics and for the degree of control it exposes over the underlying HTTP internals.

That degree of control is necessary for the fingerprinting work described above. Producing HTTP/2 fingerprints that match those of the Tor Browser requires intervening at a level below what most HTTP clients expose to the end user. Patches were applied to Hyper’s HTTP/2 implementation to allow the crawler to control the specific ordering and values of SETTINGS frames, window update behaviour, and header encoding decisions that together constitute the HTTP/2 fingerprint. Without this access, fingerprint spoofing would not be feasible. The patches sit alongside the BoringSSL modifications as part of the same build-time dependency on a patched dependency tree.

¹Though most hidden services expect that JavaScript is unavailable as it is disabled in the default Tor browser installation.

6.5.6 Arti

Rather than routing traffic through a local Tor daemon accessed via a SOCKS5 proxy², the crawler uses Arti (The Tor Project, 2021) integrated directly into the crawler process. This eliminates the inter-process socket that a SOCKS5 would require, removing a communication boundary at which logging or interception could occur. The Tor bootstrapping process is managed programmatically, which reduces start-up time and eliminates the need for a separately running services and network sockets.

During development, several deficiencies were identified within the Arti codebase. Rather than finding workarounds, fixes were contributed directly to the project, and can be observed in recent Arti changelogs (The Tor Project, 2026).

6.5.7 Rate Limiting and Politeness Controls

The crawler implements two distinct politeness mechanisms at the HTTP client level. The first is a per-authority rate limiter, which throttles requests to any single hidden service to avoid generating load that could be detected as anomalous or degrade service availability. The second is an exponential backoff system, which delays retries on connection failure or error response with progressively increasing intervals, reducing the likelihood of compounding load on an already failing service.

Both mechanisms operate at the shared HTTP client layer, which is accessed concurrently by multiple parallel processors. To ensure that politeness constraints are not silently violated under concurrency, the rate limiting state is managed using atomic intrinsics. It is worth noting that politeness inherently affects overall throughput, since enforced delays reduce the rate at which the system can make progress. This is an accepted trade-off, as the ethical obligations to direct Tor participant stakeholders take precedence over crawl speed.

Unrestricted or impolite crawling directly affects the direct Tor participant stakeholders s_P on transitions involving λ_{recv} . This is because high request rates increase the probability of service degradation and pattern detection. The rate limiters and backoff therefore reduce the total harm vector of the restricted crawler.

6.5.8 Redirect Following and Redirect Loop Detection

The crawler follows HTTP redirects up to a configurable depth and maintains a per-session visited set to detect and break redirect loops. The depth limit serves two purposes. It bounds the number of requests a single seed URL can generate, which complements the rate limiting described above, and it limits exposure to redirect chains that progressively

²SOCKS5 is Tor's default proxy interface for client applications.

lead the crawler toward content outside the intended crawl scope. The visited set ensures that any URL encountered more than once within a session is immediately abandoned, preventing a malicious hidden service from trapping the crawler in an infinite redirect cycle that would exhaust resources and generate anomalous traffic patterns. Scoping the visited set to the session rather than the entire crawl lifetime is a deliberate decision, as legitimate services may serve the same URL across independent crawl sessions without this constituting adversarial behaviour.

6.5.9 SHA-256 URL Hashing

Discovered URLs are stored in the data store only in their SHA-256 hashed form. The unhashed URL is never written to persistent storage. This means the data store cannot function as a directory of discovered hidden service addresses. A party who obtains access to the data store, including members of the research team, cannot extract a list of live onion addresses from it.

In the formal model, retaining raw URLs in the data store would constitute a property assignment that is declared forbidden. A raw onion address in plaintext in a persistent store satisfies the conditions for a forbidden state $f \in F$ in a way that directly threatens s_P and s_I . By hashing at the point of ingest, the λ_{write} trigger can only produce a state in which the URL is present in hashed form, and no transition from that state can recover the original address.

The choice of SHA-256 is rather arbitrary, and could easily be switched for any other cryptographically secure hashing algorithm.

6.5.10 Channel-Based Pipeline Architecture

The crawler is structured as a chain of three asynchronous stages connected by bounded channels. Each stage runs as an independent asynchronous task, and each connecting channel has a fixed capacity. If a downstream stage falls behind, the upstream stage is forced to wait rather than continuing to produce work that cannot yet be consumed. See Figure 6.2 above for the specific architecture. Descriptions of the pipeline architecture are intentionally kept at a high level of abstraction to discourage reproduction of the system.

Because data moves through the pipeline as discrete, typed messages, the scope of what any given stage can observe is limited to what it has been explicitly passed. This enforces a form of state exclusivity within the software itself, where each stage occupies a specific position in the transition relation and the type system prevents it from observing data that belongs to a different stage. Any change to what information crosses a stage boundary must be reflected in a type signature, which the compiler enforces and which remains visible in the version-controlled codebase.

6.5.11 OTLP Tracing with Merkle-Signed Spans

The crawler emits structured traces using the Rust `tracing` library, exported via the OpenTelemetry Protocol to a collector on Network B. Every span in the trace is signed as a node in a Merkle tree. The Merkle structure means that any ensuing modification to the trace record would invalidate the hash chain and be detectable.

The EDG requires that every technically enforced control be documentable and that the justificatory record can be verified after the fact. A conventional log can be edited without detection and therefore does not satisfy this requirement in a zero-trust framing. The Merkle signing scheme ensures that the audit record itself has integrity. If the research is later subjected to scrutiny, the trace record can demonstrate what operations were performed.

The cost of this control is explicitly acknowledged. The signing overhead reduces trace throughput by approximately half compared to an unsigned OTLP configuration. A measurable reduction in operational throughput is accepted in exchange for a cryptographically strong audit guarantee.

6.6 Summary of Controls and Framework Mappings

The table below summarises each control described in this chapter, its classification under Definition 4.3, and the formal property it primarily enforces.

Control	Class	Trigger / Transition	Required Property
Locked room, access schedule	$\mathcal{C}_T / \mathcal{C}_P$	λ_{view}	Zero-Trust Safety
VLAN A/B separation	$\mathcal{C}_T$	$\lambda_{\text{write}}, \lambda_{\text{breach}}$	Protection Monotonicity
Air-gapped PXE server	$\mathcal{C}_T$	λ_{breach}	Zero-Trust Safety
NixOS declarative config	$\mathcal{C}_T$	All transitions	Zero-Trust Safety, Well-Foundness
CIS hardening	$\mathcal{C}_T$	λ_{breach}	Zero-Trust Safety
Secure Boot	$\mathcal{C}_T$	λ_{breach}	Zero-Trust Safety
LUKS encryption	$\mathcal{C}_T$	λ_{breach}	Zero-Trust Safety
Rust memory safety	$\mathcal{C}_T$	$\lambda_{\text{breach}}, \lambda_{\text{leak}}$	Zero-Trust Safety
Zero-allocation zeroing	$\mathcal{C}_T$	$\lambda_{\text{buf}}, \lambda_{\text{breach}}$	State Exclusivity
Per-frame processing	$\mathcal{C}_T$	λ_{buf}	State Exclusivity, $\Delta\Gamma > 0$
TLS/HTTP2 fingerprint spoofing	$\mathcal{C}_T$	$\lambda_{\text{breach}}, \lambda_{\text{leak}}$	Zero-Trust Safety, $\Delta\Gamma > 0$
Hyper with applied patches	$\mathcal{C}_T$	$\lambda_{\text{breach}}, \lambda_{\text{leak}}$	Zero-Trust Safety, $\Delta\Gamma > 0$
Rate limiting and backoff	$\mathcal{C}_T$	λ_{recv}	$\Delta\Gamma > 0$
SHA-256 URL hashing	$\mathcal{C}_T$	λ_{write}	Zero-Trust Safety
Channel-based pipeline	$\mathcal{C}_T$	All transitions	State Exclusivity
Merkle-signed OTLP traces	$\mathcal{C}_T$	$\lambda_{\text{view}}, \lambda_{\text{pub}}$	Well-Foundness
Git repository, access controls	$\mathcal{C}_T / \mathcal{C}_P$	All transitions	Well-Foundness

The access schedule sits at the boundary between $\mathcal{C}_T$ and $\mathcal{C}_P$. The physical lock is a technical control, but the schedule itself is a policy commitment. Under the zero-trust framing in Section 3.1, this means that $\text{Reach}_T(\mathcal{M})$ is computed assuming the schedule could fail. The remaining $\mathcal{C}_T$ controls must therefore be sufficient to block forbidden states independently of whether the schedule is observed.

6.7 Ethical Costs

The previous section establishes that this architecture satisfies the required formal properties. This section examines what satisfying those properties costs in operational terms.

The per-frame processing constraint and the zero-allocation memory model together preclude certain categories of content-level analysis. A crawler that accumulates full page bodies before processing can apply document-level classifiers, extract structured data from the DOM (Document Object Model), or perform in-memory deduplication against a large

document corpus. The restricted design cannot do any of these without first writing content to the data store and performing analysis there. This is an accepted consequence of the reduced memory footprint and narrowed attack window that per-frame processing provides.

The Merkle-signed tracing represents the most quantifiable throughput cost. The signing overhead approximately halves the rate at which spans can be emitted. For a high-frequency crawl, this places a ceiling on trace verbosity. The decision was made to reduce trace verbosity rather than reduce signing coverage, as remaining in $\mathcal{C}_T$ was deemed a more important requirement.

TLS fingerprint spoofing introduces a dependency on patching the underlying TLS library at build time. This increases the complexity of the build toolchain and requires that the patches be maintained against upstream BoringSSL updates. The cost here is maintenance burden rather than runtime throughput.

Rate limiting and exponential backoff directly reduce crawling throughput. The alternative, an unconstrained crawl rate, would increase κ on λ_{recv} transitions for s_P . The throughput reduction is therefore a direct component of $\Delta\Gamma$.

The existence of these costs, and their explicit acknowledgement here, is what distinguishes the restricted system from one that claims ethical compliance whilst retaining the full capabilities of an unconstrained crawler.

Chapter 7

Performance

In order to validate that the ethical controls described in Chapter 6 do not render the crawler operationally inviable, this chapter evaluates the performance of a restricted crawler implementation against a comparative baseline. Crator, the open-source Tor crawler introduced by De Pascale et al. (2024), was selected because it is publicly released and sufficiently documented for independent reproducibility. Several other open-source crawlers identified in the literature rely on distributed Kubernetes orchestration or Docker clusters, which were not viable within the constrained hardware environment described in Section 6.2, and were therefore excluded from comparison. The maximum crawl depth is set to four, matching the evaluation conducted by De Pascale et al. (2024) and ensuring that results are directly comparable across both studies. The crawler operates breadth-first, consistent with the approach taken in the original paper. All figures are derived from ten repeated runs per depth using Hyperfine (Peter, 2023), with two warmup runs preceding each measurement to eliminate cold-cache effects. To validate the comparison, Crator’s published figures are reproduced independently and compared against our own measurements before any cross-system evaluation is conducted. For clarity, $\mathcal{M}_{\text{restricted}}$ is referred to as Trawler¹ throughout the remainder of this chapter.

¹Tor + crawler

7.1 Experimental Environment

All benchmarks were executed on the same netbooted NixOS environment described in Chapter 6. Because the machines boot from a PXE server over the network, the running system resides entirely in RAM. The absence of a persistent local disk has a non-trivial interaction with Crator’s architecture, which is discussed in Section 7.2. Both crawlers were pointed at the same seed URL set and run to the same maximum depth in each trial. Both systems operated under conditions that are broadly fair given the different architectural choices each makes.

7.2 A Note on Crator’s Architecture

Before presenting results, it is worth characterising Crator’s internal design, as it has a direct bearing on the interpretation of the collected metrics.

Crator is implemented in Python and follows a download-then-process architecture. Downloads are dispatched to a thread pool, and results are retrieved in the main processing loop. The critical characteristic is that all downstream work is serialised behind the download. The main loop cannot begin processing a page until the full response has been resolved by the download pool, limiting the concurrency of the system to the downloading subprocess. This is further compounded by Python’s Global Interpreter Lock, which prevents true parallel execution across threads within the same process. Each fully resolved webpage is then subsequently written to disk.

Due to the constrained netbooted and diskless environment that the software runs on, Crator’s file writes land on a tmpfs mount. This is consistent with the data minimisation principles that govern Trawler, ensuring that crawled content is never written to persistent storage. Nevertheless, tmpfs artefacts survive for as long as the system remains powered, so additional precautions were taken. A `shred` command with multiple overwrite passes was executed at the end of every trial to overwrite any remaining artefacts in memory. The device was then powered off and unplugged from the network, providing a secondary guarantee in the event that `shred` did not fully overwrite all allocated pages. None of these operations had any bearing on Crator’s performance, as tmpfs writes are fast, and `shred` operations were run between trials.

By contrast, Trawler has no such issues. The channel-based pipeline described in Section 6.5.10 is built on lower-level asynchronous primitives that allow network I/O, link extraction, and URL dispatch to proceed concurrently without contention or file writes.

7.3 Execution Time

Execution is the wall-clock duration from crawl initiation to the point at which all reachable URLs at the target depth have been processed. This is the most direct proxy for operational throughput. Raw figures are provided in Table C.1.

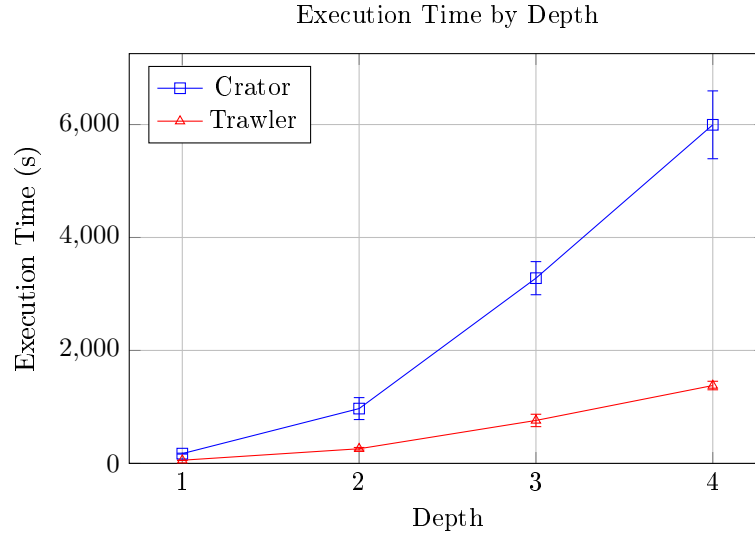


Figure 7.1: Execution time by depth (smaller is better)

Across all depths, Trawler consistently executes in significantly less time than Crator.

The variance in Crator’s depth four measurements is considerably larger than at any preceding depth. Whether this reflects an architectural limitation or Tor network noise is difficult to determine from the present data alone, though it is worth noting that Trawler shows no comparable increase in variance across the same depths, with execution time growing at a steady and measured rate throughout. Four depths are insufficient to draw firm conclusions about long-term scaling behaviour, but the diverging growth rates are suggestive of architectural inefficiencies in Crator.

7.4 Discovered URLs

Execution time alone cannot fully evaluate a crawler. A faster system that discovers significantly fewer URLs than its comparator is not a better system, it is one that is doing less work. Full URL discovery figures are provided in Table C.2.

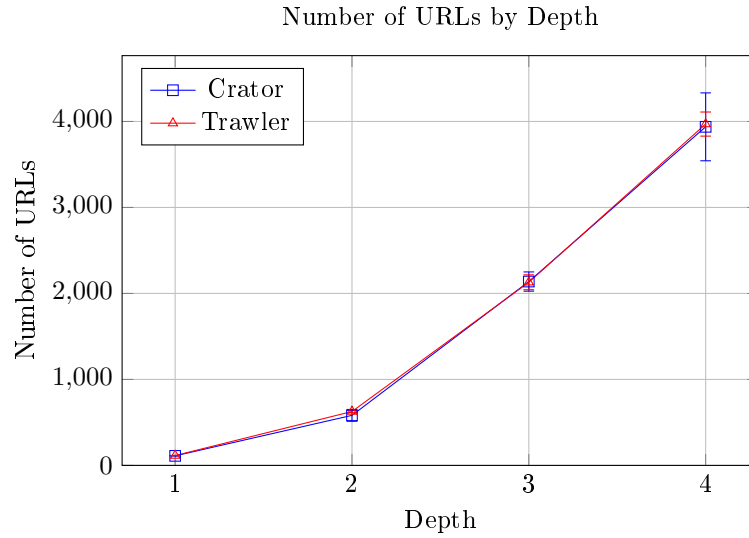


Figure 7.2: Number of discovered URLs by depth

The data show that both crawlers discover nearly identical numbers of URLs at every depth. The small variance that does exist is most likely a product of Tor’s inherent non-deterministic routing rather than any inconsistency between the crawlers. Combined with the execution time, this proves that Trawler is faster whilst maintaining the same coverage.

7.5 Pages Per Second

The pages per second data consolidate the preceding figures into a single measure of throughput. Tabulated figures are provided in Table C.3.

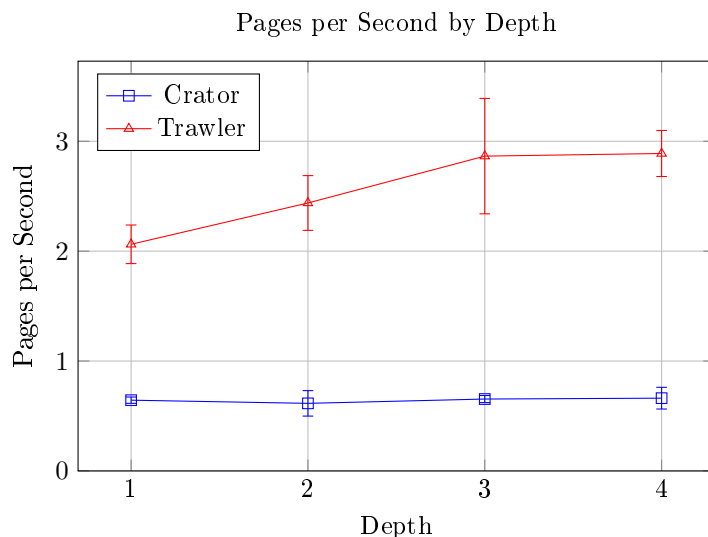


Figure 7.3: Pages per second by depth (larger is better)

Across all depths, Trawler demonstrates a throughput advantage of roughly three to four times that of Crator, with the gap remaining consistent as depth increases.

The higher standard deviation in Trawler’s results warrants some attention. Crator’s throughput variance appears low, which may reflect a rate slow enough that network jitter is largely absorbed within each run. Trawler, processing pages considerably faster, is consequently more exposed to the underlying non-determinism of Tor circuit resolution. When circuits are slow to establish, the impact on throughput is proportionally greater because more work is being attempted in the same window. The data therefore suggests that the variance is a consequence of higher throughput rather than implementation instability, which is consistent with the Tor-induced variance observed in the URL discovery metric.

7.6 Peak Memory Usage

Peak memory usage was recorded using process-level instrumentation throughout each run. Given that Rust’s memory model is a central design motivation for the implementation, this metric provides a useful empirical check against those claims. Full memory figures are provided in Table C.4.

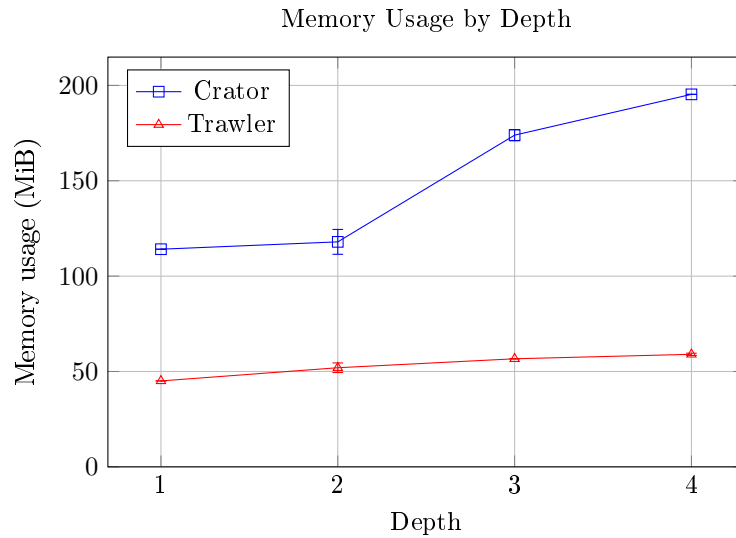


Figure 7.4: Memory Usage by Depth (smaller is better)

The Rust implementation uses substantially less memory than Crator at every depth, and scales more favourably as depth increases. This matters in practice, since a crawler that remains memory-efficient at depth can be parallelised further without a proportional increase in resource requirements. This advantage compounds with the throughput figures above, since Trawler achieves higher throughput at lower memory cost.

It should be noted that several configurations exhibit zero variance across all ten measurements, which warrants some attention. The results may reflect that Hyperfine records peak resident set size (RSS), which is a measurement quantised to the operating system’s memory page size. This implies that a crawl may stabilise at a particular allocation boundary without crossing it between runs, and that sampled peak data would therefore be identical regardless of minor internal variation. The lack of variance may also be a property of the I/O-bound nature of these systems, where the memory footprint is dominated by the crawl graph structure. Determining the true cause would require finer-grained allocation profiling, which falls outside the scope of the present evaluation.

7.7 Comparison with De Pascale et al. (2024)

The preceding results rest on the assumption that our Crator replication is a faithful reproduction of the system described in the original publication. De Pascale et al. (2024) report execution time figures for Crator, and a direct comparison between those published figures and our independently measured values allows that assumption to be tested. Tabulated figures are provided in Table C.5.

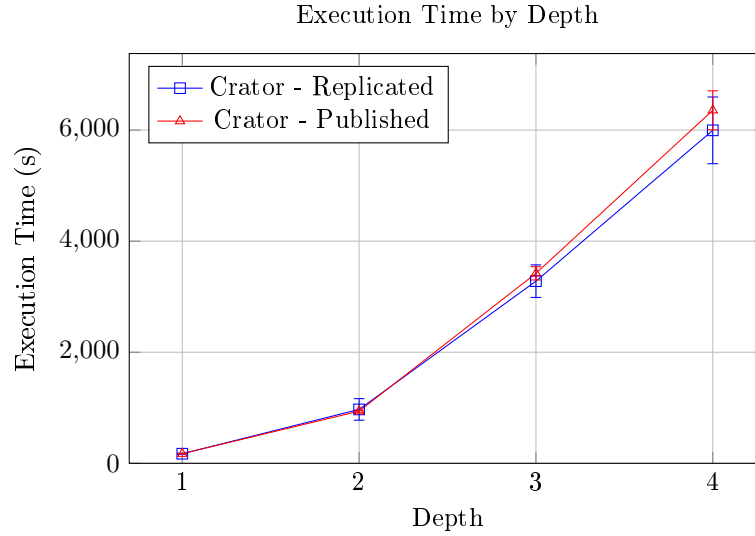


Figure 7.5: Execution time by depth (smaller is better)

Across all four depths, our replication falls within 6% of the published figures. Given the non-deterministic nature of Tor circuit selection and the variability inherent in routing requests through the network, this degree of correspondence is well within the expected range of natural variance. The results do not follow a consistent directional bias either, with our replication being faster at some depths and slower at others, which is indicative of network noise rather than any systematic difference in environment or methodology.

De Pascale et al. (2024) also report throughput figures for Crator, which provide a second point of comparison. Their figures are reported in pages per minute and are converted to pages per second for direct comparability with the results collected here. Tabulated figures are provided in Table C.6.

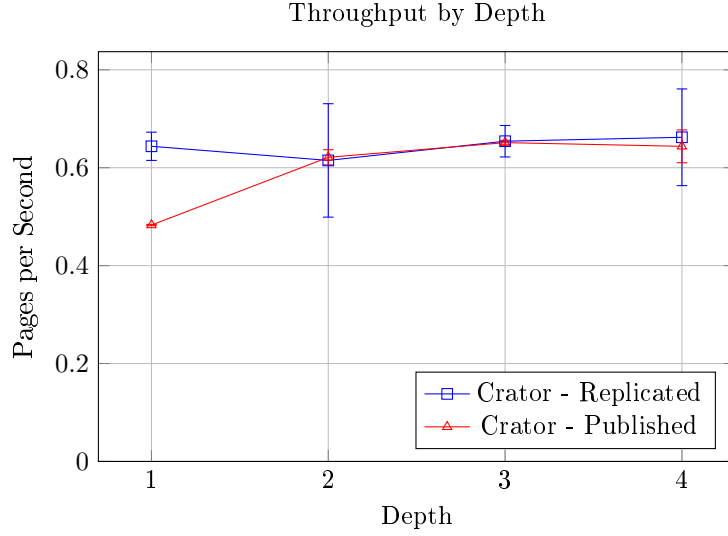


Figure 7.6: Throughput by depth (higher is better). The original values are converted from pages per minute to pages per second for comparison.

The throughput figures are largely consistent with the published values, with depths two through four falling within 3% of the figures reported by De Pascale et al. (2024) and corroborated by our own measurements. The deviation at depth one is more pronounced. Our own replication at depth one falls inline with the flat trend observed across the remaining depths, whereas De Pascale et al. (2024)’s depth one figure is considerably lower. A cold start may account for this discrepancy, as circuit establishment overhead represents a much larger proportion of total runtime when the workload is small. Our warmup runs would have left caches warm and the SOCKS5 proxy fully operational before timing began, mitigating this effect.

Both execution time and throughput metrics confirm that our replication closely models the behaviour reported by De Pascale et al. (2024), and the broader comparison between Trawler and Crator can therefore be treated as well-grounded.

7.8 Interpretations

7.8.1 Sources of the Throughput Difference

The most immediate explanation for the execution time advantage observed in Trawler is the use of a compiled, statically typed language. Rust code is compiled to native machine code with no interpreter overhead and no garbage collector pauses. Python, carries interpreter overhead on every bytecode dispatch, and is subject to the Global Interpreter Lock, which limits true parallelism within a single process.

However, attributing the entire advantage to language choice would be an oversimplification, and potentially an incorrect one. De Pascale et al. (2024) themselves compare Crator’s performance against ACHE (Barbosa & Freire, 2007), a crawler written in Java. They demonstrate that Crator significantly outperforms ACHE despite Java’s native compilation advantage over Python. Their analysis attributes Crator’s advantage to architectural and algorithmic choices rather than to the implementation language alone.

Applying the same logic to the present comparison, a significant part of our throughput advantage likely derives from the streaming extraction architecture rather than from Rust per se. A Python implementation of the same streaming architecture would likely retain the interpreter overhead but would also eliminate the download-then-process pattern. The relative contributions of language and architecture cannot be fully untangled from the present data alone, however De Pascale et al. (2024)’s comparison with ACHE provides reason to weigh the architectural factor heavily.

The architectural difference also has implications beyond raw throughput. Crator’s download-then-process design introduces a scalability ceiling that is independent of network speed. The total data written to the intermediate file store grows with the number of URLs processed, which increases exponentially with crawl depth. This places a hard upper limit on the depth at which Crator can complete a run without exhausting available storage. The proposed crawler does not face this constraint, as writes are bounded by the number of unique URLs rather than total page content, meaning its storage footprint scales considerably more favourably at depth.

7.8.2 Ethical Overhead

The throughput advantage observed here is achieved despite the proposed crawler performing substantially more work per URL than Crator. For every URL processed, Trawler computes a SHA-256 hash of the URL before any write occurs, emits a structured OpenTelemetry span signed as a node in a Merkle tree to the collector on Network B, commits a record to the append-only data store, and enforces politeness delays and exponential backoff per domain. None of these operations are present in Crator.

It is worth noting that Merkle span signing does not affect wall-clock crawl time directly. The signing operation runs on a dedicated background thread and is therefore off the critical path of URL dispatch and extraction. Politeness, SHA-256 hashing, and database writes do add synchronous work for every URL that would otherwise be absent, meaning Trawler performs substantially more work per request than Crator.

It is worth observing that decisions made in the planning phase had a notable impact on performance beyond their primary purpose. The obligation to minimise raw content at the earliest opportunity likely reduces the volume of data processed per crawled page, which would contribute to an increased throughput. In this respect, careful ethical design choices served as an optimisation rather than an impedance.

7.9 Summary

The results converge on a clear conclusion. Trawler processes pages significantly faster than Crator at every depth, and does so whilst maintaining equivalent coverage. The replication of Crator’s published figures provides confidence that this comparison is well-grounded. We achieved this despite Trawler performing substantially more work per URL than Crator, and whilst bearing the weight of the ethical controls described in Chapter 4. This demonstrates that verifiable ethical compliance and practical utility are not mutually exclusive objectives.

Chapter 8

Conclusion

8.1 Future Work

The constraints imposed by this framework are deliberate, but they are not without cost. The decision to minimise raw content as early as technically possible substantially narrows the analytical work that the crawler can do. This is an accepted trade-off and it points to a natural progression for future research. The question is not how to relax the constraints, but whether the same analytical goals can be achieved without violating them.

One of the more promising directions comes from the field of privacy-preserving machine learning. For image-based content, a crawler operating under this framework could sample every n th pixel rather than retaining a full image payload. The reduced representation would retain enough broad structural information to be of analytical use, whilst substantially reducing what could constitute a privacy harm if exposed. Whether a classification model can operate reliably on these highly modified inputs is an open empirical question.

For text-based analysis, Lee et al. (2022) have already demonstrated that inference over encrypted BERT representations is feasible, allowing for semantic classification without exposing the underlying text to the model host. Integrating such a pipeline into the data transformation stage described in Chapter 4 would allow for richer content classification. Finding ways to meaningfully analyse data that remains encrypted at rest and in use represents an extremely valuable avenue for extending this work.

A separate and more practically immediate limitation is the inability to access authenticated content. Much of the hidden service landscape sits behind login flows, session cookies, and CAPTCHA challenges. The approaches most commonly used in the literature, such as cookie extraction or Selenium-based browser automation, both create exposure paths that this framework is designed to prevent. A researcher manually acquiring cookies must

view authenticated content to do so, which triggers the forbidden state f_1 . Automated browser rendering has the same problem unless the rendering environment is very carefully controlled.

A more principled solution would be a patched or instrumented browser that can acquire session credentials whilst blocking content rendering. The researcher could operate the authenticated flow without being able to perceive the content behind it, and the credentials could then be handed to the automated crawler. Whether this is achievable in a way that is genuinely tamper-resistant and whether it is compatible with the non-interaction commitments described in Chapter 2 are left undiscussed.

8.2 Conclusion

Ethical discussion in empirical Tor research tends to be relatively lacking. Where it appears, it tends to function as a statement of approval rather than a record of reasoning, focusing on compliance rather than the substance of the decisions being made. Vulnerable populations are rarely considered, and researcher wellbeing is almost never addressed. The release of tools and datasets is treated as routine, with little thought for what they might enable in the hands of others. This is a genuine problem for a field that operates in close proximity to some of the most sensitive and potentially harmful material that researchers are likely to encounter.

The response developed here draws from the zero-trust principle, familiar from information security. Safe outcomes cannot be made to depend on the good behaviour of individuals, because individuals are fallible. If a research system can reach a state where raw content is directly viewable, that state will eventually be reached. Therefore a system should make its best efforts to make these states unreachable, rather than discouraged. This shifts the ethical work away from the researcher to the system itself, which promotes verification as opposed to trust.

The Ethical Dependency Graph and State Transition Harm model formalise this intuition into a system that can be reasoned with. They provide a way to declare the forbidden states, enumerate controls to prevent such states, and verify that those controls are sufficient. The comparison between naive crawler and the restricted crawler implemented in this paper makes the affect of these controls observable. The restricted system does not reach forbidden states, and the reasoning behind each practical decision can be traced to a specific control made in the planning phase.

These decisions come with real costs, and this paper has not tried to minimise them. The audit overhead, and narrowed scope of analysis are genuine limitations which are a direct consequence of the constraints the framework imposes. It would be a false claim for

any ethical framework to state that it has no practical cost. The hope is that the approach developed here is useful not because it is the only way to do ethical Tor research, but because it demonstrates that rigorous ethical constraints and functional research systems are compatible without hand-wavy moral postulates.

The performance evaluation reported in Chapter 7 gives our claim its empirical grounding. The restricted crawler completes faster at every tested depth than an unrestricted comparator whilst maintaining equivalent coverage, and it does so whilst performing considerably more work per URL. This is perhaps the most direct rebuttal available to the assumption that ethical constraints and a functional research system are fundamentally at odds.

It is worth stating plainly that nothing in this framework is specific to the dark web. The underlying ethical challenges are present wherever researchers collect data pertaining to real people in real contexts. Tor does not create these problems, it simply concentrates and intensifies them due to the nature of the network. In that sense, Tor is a useful proving ground for this genre of research. If a framework can hold under these conditions, it is likely to hold in less extreme ones. Treating ethics as an architectural property rather than a procedural one may prove useful well beyond this setting.

The literature’s evasion of ethics is not a neutral omission. Choosing not to examine the ethical character of a research system is still a choice, and the pattern observed across the reviewed corpus is difficult to explain except as an effort to avoid the cognitive overhead that genuine deliberation demands. Whilst this overhead is a valid concern, it is not the researcher’s alone to dismiss. Those whose privacy and security are implicated in that decision were never given the same privilege.

Bibliography

- Bailey, M., Dittrich, D., Kenneally, E., & Maughan, D. (2012). The Menlo Report. *IEEE Security & Privacy*, 10, 71–75. <https://doi.org/10.1109/MSP.2012.52>
- Barbosa, L., & Freire, J. (2007). An adaptive crawler for locating hiddenwebentry points. *16th International World Wide Web Conference, WWW2007*, 441–450. <https://doi.org/10.1145/1242572.1242632>
- Benjamin, V., Valacich, J. S., & Chen, H. (2019). Dice-e: A framework for conducting dark-net identification, collection, evaluation with ethics. *MIS Quarterly: Management Information Systems*, 43, 1–22. <https://doi.org/10.25300/MISQ/2019/13808>
- Bergman, J., & Popov, O. B. (2023). Exploring dark web crawlers: A systematic literature review of dark web crawlers and their implementation. *IEEE Access*, 11, 35914–35933. <https://doi.org/10.1109/ACCESS.2017.DOI>
- Brewer, R., Westlake, B., Arauza, O., & Hart, T. (2021, January). The ethics of web crawling and web scraping in cybercrime research: Navigating issues of consent, privacy, and other potential harms associated with automated data collection. In *Researching cybercrimes: Methodologies, ethics, and critical approaches* (pp. 435–456). Springer International Publishing. https://doi.org/10.1007/978-3-030-74837-1_22
- British Society of Criminology. (2015). *Statement of ethics* (tech. rep.). <http://www.britsoccrim.org/ethics/>
- Celestini, A., & Guarino, S. (2017). Design, implementation and test of a flexible tor-oriented web mining toolkit. *ACM International Conference Proceeding Series, Part F129475*. <https://doi.org/10.1145/3102254.3102266>
- CIS Center for Internet Security. (2017). CIS Independent Linux Benchmark v2.0.0. <https://www.cisecurity.org/cis-benchmarks>
- Dalins, J., Wilson, C., & Carman, M. (2018). Criminal motivation on the dark web: A categorisation model for law enforcement. *Digital Investigation*, 24, 62–71. <https://doi.org/10.1016/j.diin.2017.12.003>
- David, B., Delong, M., & Filiol, E. (2021). Detection of crawler traps: Formalization and implementation—defeating protection on internet and on the tor network. *Journal*

- of *Computer Virology and Hacking Techniques*, 17, 185–198. <https://doi.org/10.1007/s11416-021-00380-4>
- De Pascale, D., Cascavilla, G., Tamburri, D. A., & Heuvel, W.-J. V. D. (2024). Crator: A dark web crawler. <http://arxiv.org/abs/2405.06356>
- Denning, D. E. (1976). *A lattice model of secure information flow* (tech. rep.). Purdue University.
- Google. (2014). Boringssl. <https://boringssl.googlesource.com/boringssl/>
- Guitton, C. (2013). A review of the available content on tor hidden services: The case against further development. *Computers in Human Behavior*, 29, 2805–2815. <https://doi.org/10.1016/j.chb.2013.07.031>
- Kang, D. W., & Kim, J. (2026). Identification of criminal networks via external identifier-based clustering on the dark web. *IEEE Access*. <https://doi.org/10.1109/ACCESS.2026.3658951>
- Kuhn, P., Wittorf, K., & Reuter, C. (2024). Navigating the shadows: Manual and semi-automated evaluation of the dark web for cyber threat intelligence. *IEEE Access*, 12, 118903–118922. <https://doi.org/10.1109/ACCESS.2024.3448247>
- LaPadula, L. J., & Bell, D. E. (1996, November). *Secure computer systems: Mathematical foundations* (tech. rep.).
- Lee, G., Kim, M., Park, J. H., Hwang, S.-w., & Cheon, J. H. (2022). Privacy-preserving text classification on bert embeddings with homomorphic encryption. <http://arxiv.org/abs/2210.02574>
- McArthur, S. (2022). Hyper. <https://github.com/hyperium/hyper>
- Monterrubio, S. M. M., Naranjo, J. E. A., Lopez, L. I. B., & Caraguay, A. L. V. (2021). Black widow crawler for tor network to search for criminal patterns. *Proceedings - 2021 2nd International Conference on Information Systems and Software Technologies, ICI2ST 2021*, 108–113. <https://doi.org/10.1109/ICI2ST51859.2021.00023>
- Moore, D., & Rid, T. (2016). Cryptopolitik and the darknet. *Survival*, 58, 7–38. <https://doi.org/10.1080/00396338.2016.1142085>
- Park, J., Mun, H., & Lee, Y. (2018). Improving tor hidden service crawler performance. *2018 IEEE Conference on Dependable and Secure Computing (DSC)*, 1–8. <https://doi.org/10.1109/DESEC.2018.8625103>
- Pastor-Galindo, J., Mármol, F. G., & Pérez, G. M. (2023). On the gathering of tor onion addresses. *Future Generation Computer Systems*, 145, 12–26. <https://doi.org/10.1016/j.future.2023.02.024>
- Pastor-Galindo, J., Sandlin, H.-Â., Mármol, G., Bovet, G., & Pérez, G. M. (2024). A big data architecture for early identification and categorization of dark web sites. *Future Generation Computer Systems*, 157. <https://doi.org/10.17632/9nmpmf5>

- Peter, D. (2023). Hyperfine. <https://github.com/sharkdp/hyperfine>
- Posey, C., Bennett, R. J., & Roberts, T. L. (2011). Understanding the mindset of the abusive insider: An examination of insiders' causal reasoning following internal security changes. *Computers and Security*, 30, 486–497. <https://doi.org/10.1016/j.cose.2011.05.002>
- Rao, U. N. P., Guruprasad, R. R., Shetty, O. J., Sarasvathi, V., & Rapate, G. S. (2024). Data analysis of dark web marketplaces using machine learning. *2024 7th International Conference on Signal Processing and Information Security, ICSPIS 2024*. <https://doi.org/10.1109/ICSPIS63676.2024.10812633>
- Ródenas, J. M. R., Pastor-Galindo, J., & Mármol, F. G. (2024). A general and modular framework for dark web analysis. *Cluster Computing*, 27, 4687–4703. <https://doi.org/10.1007/s10586-023-04189-2>
- Sabbah, T., Selamat, A., Selamat, M. H., Ibrahim, R., & Fujita, H. (2016). Hybridized term-weighting method for dark web classification. *Neurocomputing*, 173, 1908–1926. <https://doi.org/10.1016/j.neucom.2015.09.063>
- Sambath, M., Maheswari, G., Prahadeesh, V., Kumar, S. P., & Sree, K. S. (2024). Torquest (the onion router crawler). *Proceedings of 9th International Conference on Science, Technology, Engineering and Mathematics: The Role of Emerging Technologies in Digital Transformation, ICONSTEM 2024*. <https://doi.org/10.1109/ICONSTEM60960.2024.10568905>
- The Tor Project. (2021). Arti: A Rust implementation of Tor. <https://gitlab.torproject.org/tpo/core/arti>
- The Tor Project. (2026). Arti Changelog. https://blog.torproject.org/arti_2_0_0_released/
- Turk, K., Pastrana, S., & Collier, B. (2020). A tight scrape: Methodological approaches to cybercrime research data collection in adversarial environments. *Proceedings - 5th IEEE European Symposium on Security and Privacy Workshops, Euro S and PW 2020*, 428–437. <https://doi.org/10.1109/EuroSPW51379.2020.00064>
- Xu, Y., Chen, G., Wu, J., Xu, W., & Liu, Q. (2021). Research on dark web monitoring crawler based on tor. *Proceedings of 2021 IEEE 2nd International Conference on Information Technology, Big Data and Artificial Intelligence, ICIBA 2021*, 197–202. <https://doi.org/10.1109/ICIBA52610.2021.9687954>

Appendix A

Supplementary Tables

A.1 Decision Set

ID	Decision	Stage ϕ	$E(d)$	$M(d)$	$\mathcal{A}(d)$
d_1	Crawl Tor hidden services for empirical research	ϕ_s	(3, 2, 2, 2, 1)	(0, 0, 0, 0, 0)	c_{16}
d_2	Use Rust and Arti	ϕ_s	(0, 0, 0, 0, 0)	(1, 0, 1, 0, 0)	c_1, c_8
d_3	Avoid browser emulation	ϕ_s	(0, 0, 0, 0, 0)	(1, 1, 0, 1, 0)	—
d_4	Obtain ethical and institutional approval	ϕ_p	(0, 0, 0, 0, 0)	(1, 1, 1, 1, 0)	c_{16}
d_5	Researchers must be isolated from all raw data	ϕ_p	(0, 0, 0, 0, 0)	(2, 2, 2, 2, 1)	c_7, c_9, c_{10}
d_6	Filter content before human interaction	ϕ_p	(0, 0, 0, 0, 0)	(2, 1, 1, 1, 0)	c_2, c_9
d_7	Access to the system must be auditable	ϕ_p	(0, 0, 0, 0, 0)	(2, 1, 2, 2, 0)	c_{11}, c_{12}, c_{13}
d_8	Stored data must be minimised	ϕ_p	(0, 0, 0, 0, 0)	(2, 1, 1, 1, 0)	c_1, c_4
d_9	Zeroize memory with a zero-allocator	ϕ_i	(0, 0, 0, 0, 0)	(1, 0, 1, 0, 0)	c_1
d_{10}	HTTP HEAD request to check content type	ϕ_i	(0, 0, 0, 0, 0)	(2, 1, 1, 1, 0)	c_2
d_{11}	Never accumulate page content whilst extracting	ϕ_i	(0, 0, 0, 0, 0)	(3, 2, 2, 1, 1)	c_9
d_{12}	Store only hashed URL identifiers	ϕ_i	(0, 0, 0, 0, 0)	(3, 2, 1, 1, 1)	c_4
d_{13}	Run crawler on a diskless netbooted machine	ϕ_i	(0, 0, 0, 0, 0)	(2, 1, 2, 2, 0)	c_7
d_{14}	Use a declaratively hardened OS image	ϕ_i	(0, 0, 0, 0, 0)	(2, 0, 1, 2, 0)	c_{11}, c_{12}
d_{15}	Full disk encryption on all persistent storage	ϕ_i	(0, 0, 0, 0, 0)	(2, 1, 1, 2, 0)	c_5
d_{16}	Database on a, network-isolated machine	ϕ_i	(0, 0, 0, 0, 0)	(2, 1, 1, 2, 0)	c_3, c_6
d_{17}	Air-gap the environment via a dedicated VLAN	ϕ_i	(0, 0, 0, 0, 0)	(2, 1, 2, 2, 0)	c_6
d_{24}	Use OpenTelemetry for audit logging	ϕ_i	(0, 0, 0, 0, 0)	(0, 0, 1, 2, 0)	c_{13}
d_{18}	Merkle-tree hash audit entries	ϕ_o	(0, 0, 0, 0, 0)	(0, 0, 1, 2, 0)	c_{13}
d_{19}	Physical entry sheets for all researcher access to the secure environment	ϕ_o	(0, 0, 0, 0, 0)	(0, 0, 1, 1, 0)	c_{14}
d_{20}	Researchers sign and adhere to a code of conduct	ϕ_o	(0, 0, 0, 0, 0)	(0, 0, 0, 1, 0)	c_{15}
d_{23}	Write only filtered and hashed records into the isolated database	ϕ_o	(0, 0, 0, 0, 0)	(2, 1, 1, 1, 0)	c_3, c_4, c_6, c_{10}
d_{21}	Publish only aggregate metrics	ϕ_b	(0, 0, 0, 0, 0)	(2, 2, 0, 1, 1)	c_{15}, c_{16}
d_{22}	Destroy data artefacts once research concludes	ϕ_r	(0, 0, 0, 0, 0)	(1, 1, 1, 1, 0)	c_{13}, c_{14}, c_{15}

Table A.1: Decision set D for the restricted crawler EDG. Each decision is a tuple $d = (\iota, \phi, E(d), M(d), \mathcal{A}(d))$, where $\phi \in \Phi$ is the lifecycle stage, $E(d) \in \mathcal{K}$ is the exposure vector introduced, $M(d) \in \mathcal{K}$ is the mitigation provided, and $\mathcal{A}(d) \subseteq \mathcal{C}$ is the set of enforcing controls. Vectors are over $\mathcal{S} = \{s_P, s_I, s_R, s_N, s_X\}$.

A.2 Dependency Edges

Edge	Type	Interpretation
$d_1 \rightarrow d_4$	τ_e	Research is permissible only once ethical and institutional approval is secured
$d_1 \rightarrow d_5$	τ_e	The exposure crawling introduces obligates researcher isolation by design
$d_1 \rightarrow d_6$	τ_e	The exposure crawling introduces obligates content filtering by design
$d_1 \rightarrow d_7$	τ_e	The exposure crawling introduces obligates hardened and auditable access by design
$d_1 \rightarrow d_8$	τ_e	The exposure crawling introduces obligates maximal data minimisation by design
$d_2 \rightarrow d_9$	τ_d	The choice of Rust discharges some memory safety obligations
$d_3 \rightarrow d_{11}$	τ_d	Avoiding browser emulation discharges the risk that full page content is rendered
$d_4 \rightarrow d_{14}$	τ_e	Institutional approval requires that the operating environment is hardened and verified
$d_4 \rightarrow d_{20}$	τ_e	Institutional approval establishes the scope at which a code of conduct is enforceable
$d_5 \rightarrow d_{13}$	τ_e	Researcher isolation requires that no persistent local storage exists on the crawler machine
$d_6 \rightarrow d_{10}$	τ_e	Content filtering requires a pre-fetch content-type check to avoid non-textual payloads
$d_6 \rightarrow d_{11}$	τ_e	Content filtering requires that extraction is incapable of accumulating raw page content
$d_7 \rightarrow d_{15}$	τ_e	Hardened access requires full disk encryption on all persistent storage
$d_7 \rightarrow d_{16}$	τ_e	Hardened access requires persistent data to reside on a network-isolated machine
$d_7 \rightarrow d_{17}$	τ_e	Hardened access requires that the environment is air-gapped and network-isolated
$d_7 \rightarrow d_{24}$	τ_e	The requirement for auditable access is discharged by the use of OpenTelemetry
$d_8 \rightarrow d_9$	τ_e	Data minimisation requires that memory carrying raw content is zeroed on deallocation
$d_8 \rightarrow d_{12}$	τ_e	Data minimisation requires that no raw URL or host identifier is ever written to storage
$d_{11} \rightarrow d_{23}$	τ_e	The extractor's href-only output makes storage permissible
$d_{12} \rightarrow d_{23}$	τ_c	URL and host hashing constrains artefacts are written
$d_{13} \rightarrow d_{16}$	τ_c	Diskless operation forecloses local database storage
$d_{13} \rightarrow d_{17}$	τ_c	Diskless operation constrains the crawling environment's network boundary
$d_{14} \rightarrow d_{15}$	τ_c	A declaratively hardened image mandates full disk encryption
$d_{16} \rightarrow d_{23}$	τ_c	The append-only isolated database constrains the storage boundary
$d_{17} \rightarrow d_{16}$	τ_c	Network isolation constrains where the database may reside
$d_{24} \rightarrow d_{18}$	τ_e	The OpenTelemetry instrumentation enables Merkle-tree audit logging
$d_{18} \rightarrow d_{19}$	τ_e	Merkle-tree integrity is meaningful only when verified against an independent record
$d_{18} \rightarrow d_{22}$	τ_e	Verified destruction is auditable only if the Merkle log survives the purge
$d_{19} \rightarrow d_{22}$	τ_e	Verified destruction requires physical record to prevent further changes
$d_{23} \rightarrow d_{21}$	τ_e	Publication is permissible because no raw content is ever stored

Table A.2: Key dependency edges in $\mathcal{G}$. An enabling edge (τ_e) means one decision is permissible only because another holds. A discharging edge (τ_d) means one decision absorbs the exposure introduced by another. A constraining edge (τ_c) means one decision narrows the options available to another.

A.3 Reversal Cost Analysis

Decision Reversed	$ \text{cas}(d_i) $	Cascade Set	Harm Re-Exposed	Interpretation
d_7 (hardened auditable access)	7	$d_{15}, d_{16}, d_{17}, d_{24}, d_{18}, d_{19}, d_{22}$	$(2, 1, 2, 2, 0)$	Every isolation, auditing and hardening step derives from this decision
d_6 (content filtering)	4	$d_{10}, d_{11}, d_{23}, d_{21}$	$(2, 1, 1, 1, 0)$	Invalidates all other filtering and publication decisions if reversed
d_{24} (OpenTelemetry)	3	d_{18}, d_{19}, d_{22}	$(0, 0, 1, 2, 0)$	Removes basis of Merkle tree logging and auditable destruction
d_{11} (streaming extractor)	2	d_{23}, d_{21}	$(2, 1, 2, 1, 0)$	Raw page content accumulates in memory. f_1 and f_4 become reachable
d_{18} (Merkle audit log)	2	d_{19}, d_{22}	$(0, 0, 1, 2, 0)$	Log tampering becomes undetectable
d_8 (data minimisation)	2	d_9, d_{12}	$(2, 1, 1, 1, 0)$	Memory zeroing and hashing are invalidated
d_{23} (storage boundary)	1	d_{21}	$(2, 1, 1, 1, 0)$	Raw or unhashed content may reach storage or publication
d_5 (researcher isolation)	1	d_{13}	$(2, 2, 2, 2, 1)$	Highest harm vector. Researchers now potentially exposed to harm
d_{12} (URL hashing)	0	—	$(3, 2, 1, 1, 1)$	Highest s_P re-exposure. Database becomes linkable to artefacts.
d_{13} (diskless net boot)	0	—	$(2, 1, 2, 2, 0)$	Local storage restored. Data isolation no longer holds.

Table A.3: Reversal cost analysis for key decisions in $\mathcal{G}$. $|\text{cas}(d_i)|$ counts decisions invalidated through τ_e edges when d_i is removed; harm re-exposed is $M(d_i)$ over $\mathcal{S} = \{s_P, s_I, s_R, s_N, s_X\}$.

Appendix B

Supplementary Diagrams

B.1 State Transition Harm Model

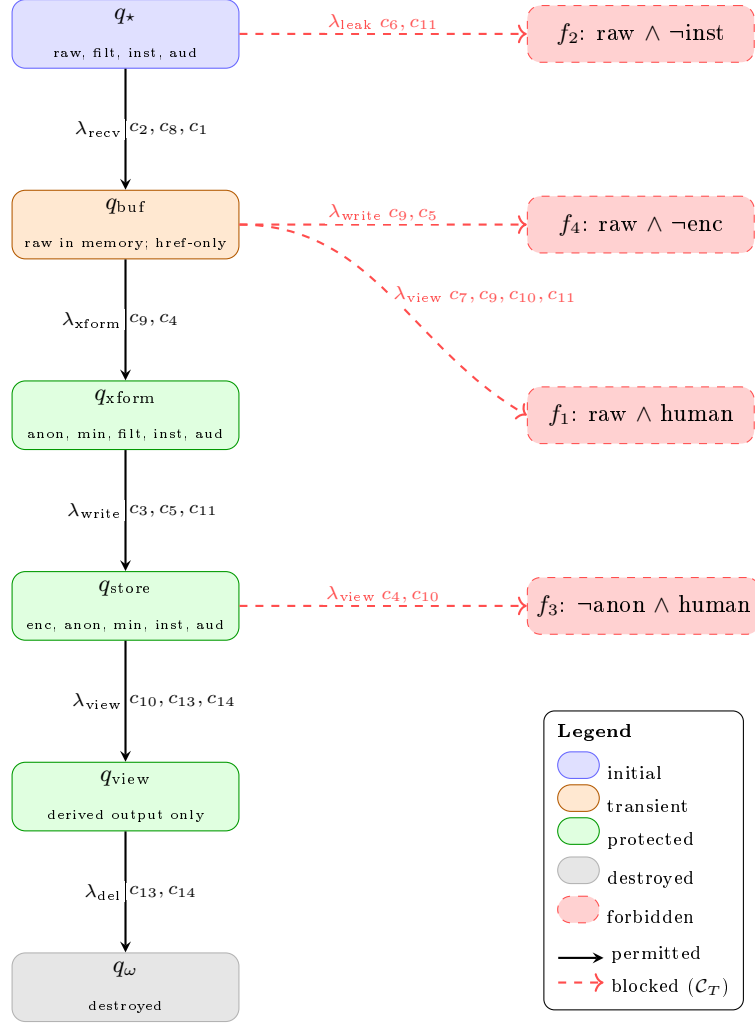


Figure B.1: State transition harm model for the restricted crawler ($\mathcal{M}_{\text{restricted}}$). States reflect the actual property sets declared in $\mathcal{P}$. Solid arrows show permitted data lifecycle transitions and their assigned controls. Dashed red arrows indicate transitions to forbidden states that are structurally blocked by technically enforced controls ($\mathcal{C}_T$). No sequence of unblocked transitions reaches any forbidden state f_i , satisfying Zero-Trust Safety (4.2.4).

B.2 Ethical Dependency Graph

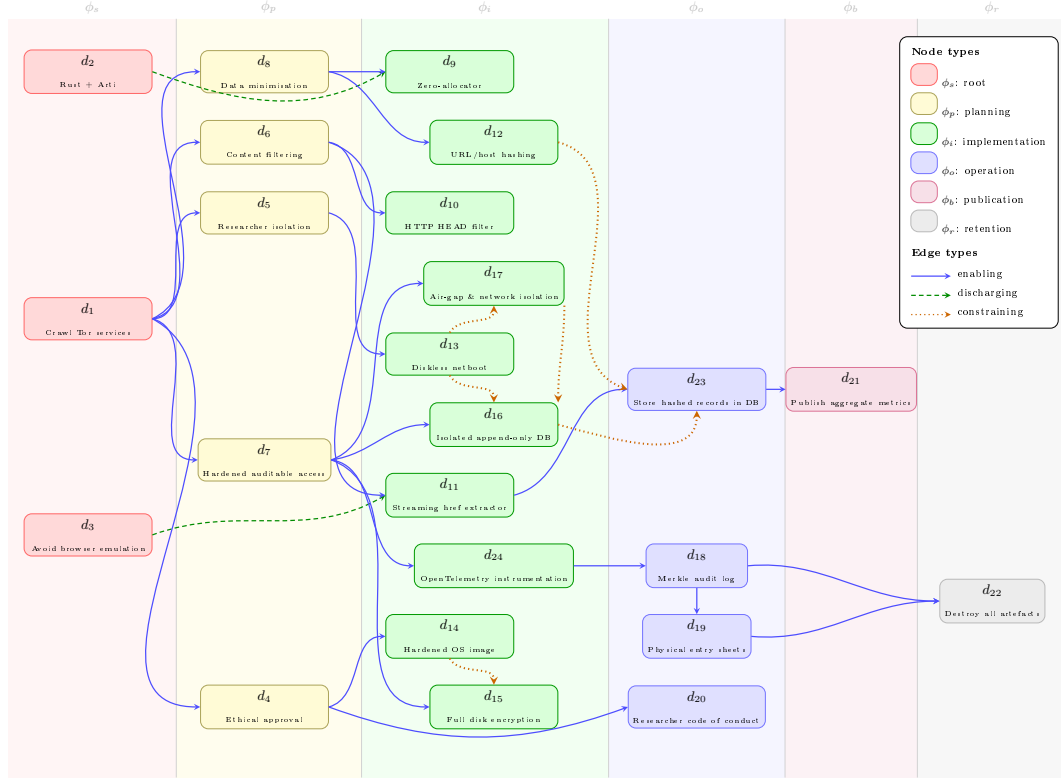


Figure B.2: Ethical dependency graph for the restricted crawler, arranged in strict lifecycle-stage bands ($\phi_s \rightarrow \phi_p \rightarrow \phi_i \rightarrow \phi_o \rightarrow \phi_b \rightarrow \phi_r$). Every edge points forward or laterally within a band and never backwards. By Proposition 4.4, this implies well-foundedness (acyclicity).

B.3 Crawled Graph

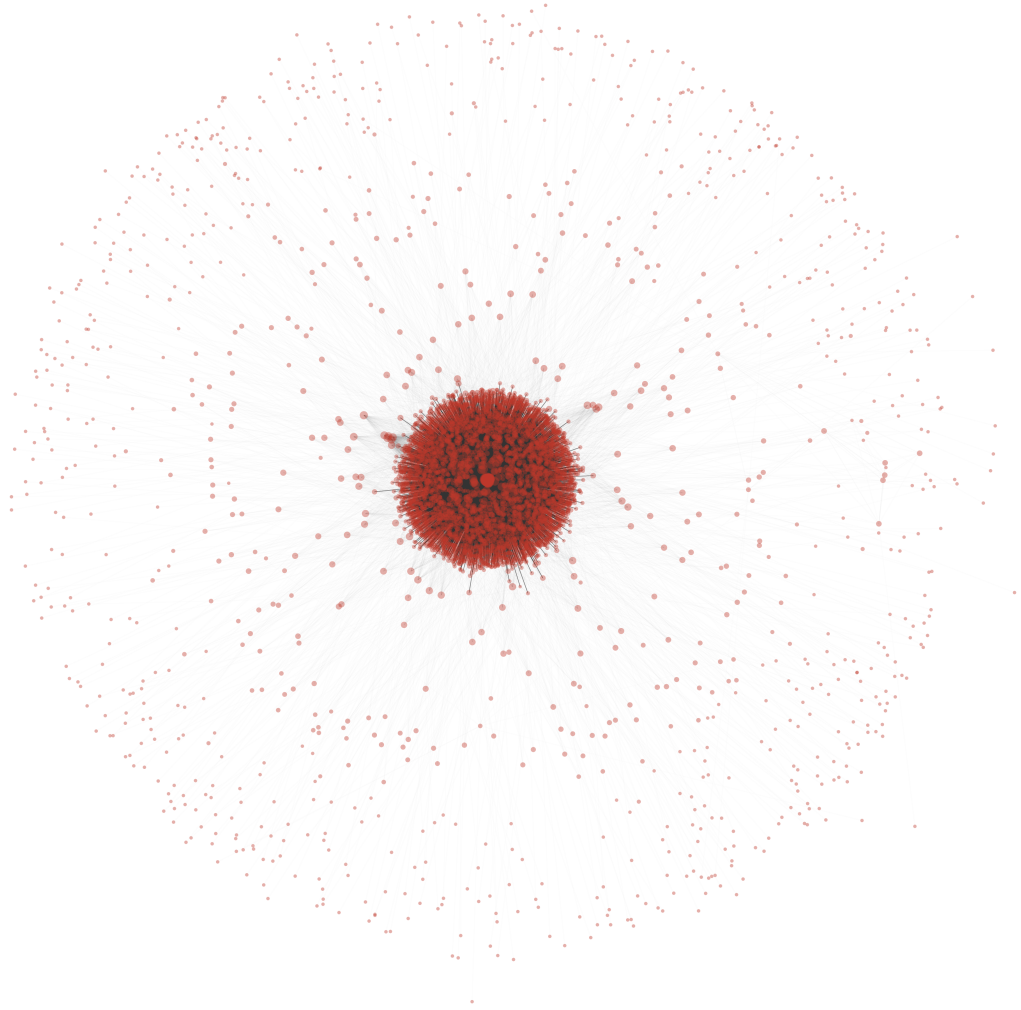


Figure B.3: Graph structure of pages discovered during a breadth-first crawl from a seed URL. Each node represents a unique page, with node size proportional to in-degree (the number of pages linking to it). Edges represent hyperlinks between pages. Some peripheral edges are difficult to discern due to the low edge opacity chosen to prevent visual saturation in the densely connected core.

Appendix C

Raw Benchmark Results

C.1 Execution Time

Depth	Mean (s)	Std Dev	Depth	Mean (s)	Std Dev
1	55.97	5.32	1	172.75	8.31
2	259.08	22.56	2	970.35	194.12
3	760.39	109.62	3	3,279.66	293.11
4	1,378.59	75.5	4	5,995.29	600.67

(a) Trawler

(b) Crator

Table C.1: Execution time by depth (seconds).

C.2 Discovered URLs

Depth	Mean	Std Dev	Depth	Mean	Std Dev
1	114.6	0.8	1	111	0
2	626.8	12.6	2	582	68.6
3	2,129.8	86.6	3	2,138.3	111.9
4	3,969.8	140.1	4	3,937.7	394.8

(a) Trawler

(b) Crator

Table C.2: URLs discovered by depth.

C.3 Pages Per Second

Depth	Mean	Std Dev	Depth	Mean	Std Dev
1	2.06	0.18	1	0.64	0.03
2	2.44	0.25	2	0.61	0.12
3	2.86	0.53	3	0.65	0.03
4	2.89	0.21	4	0.66	0.1

(a) Trawler

(b) Crator

Table C.3: Pages per second by depth.

C.4 Memory Usage

Depth	Mean (MiB)	Std Dev	Depth	Mean (MiB)	Std Dev
1	45.1	0	1	114.1	0
2	52	2.5	2	118	6.5
3	56.7	0	3	173.9	2.9
4	59	0.6	4	195.3	0

(a) Trawler

(b) Crator

Table C.4: Peak memory usage by depth (MiB).

C.5 Replicated Execution Time

Depth	Published Mean (s)	Replicated Mean (s)	Diff. (%)
1	173.3	172.7	−0.3 %
2	939.2	970.3	+3.3 %
3	3,420.6	3,279.7	−4.1 %
4	6,356.2	5,995.3	−5.7 %

Table C.5: Execution time (seconds): our Crator replication vs. figures reported by De Pascale et al. (2024).

C.6 Replicated Throughput

Depth	Published Mean	Replicated Mean	Diff. (%)
1	0.48	0.64	+33.3 %
2	0.62	0.61	−1 %
3	0.65	0.65	+0.4 %
4	0.64	0.66	+2.9 %

Table C.6: Throughput (pages per second): our Crator replication vs. figures reported by De Pascale et al. (2024). Published figures are converted from pages per minute.